Tribhuvan University

Faculty of Humanities and Social Sciences

Masters of Computer Application (MCA)

1st Semester

Advanced Database Management System(MCA-503)

Unit-4: Distributed Database Concepts 9hrs

Instructor

Tekendra Nath Yogi (M.SC CSIT TU)

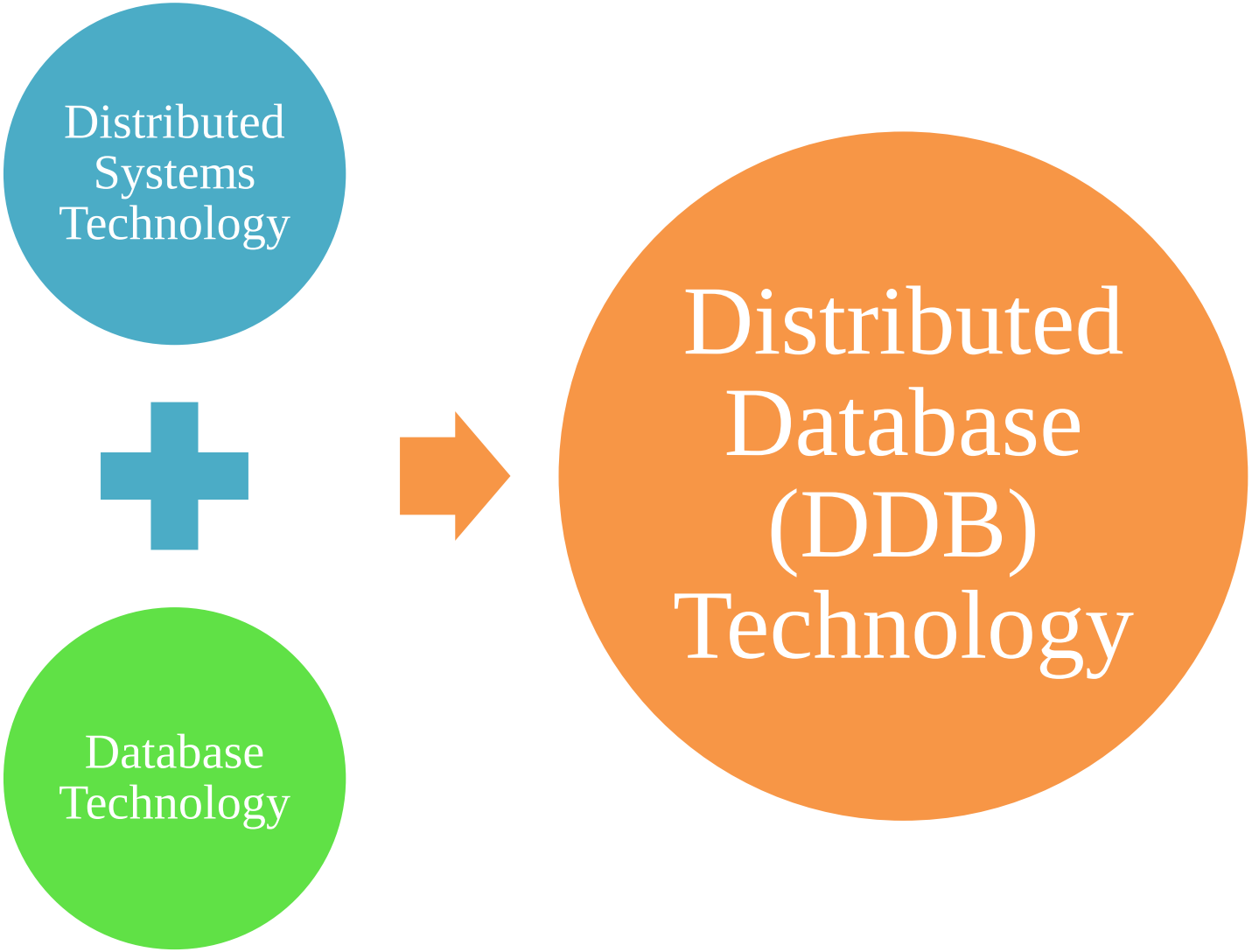
Tekendranath@gmail.com

Contents

- **Unit 4: Distributed Database Concepts [9 Hrs.]**
  - 4.1. Distributed Database Concepts;
  - 4.2. Data Fragmentation, Replication and Allocation Techniques for Distributed Database Design;
  - 4.3. Overview of Concurrency Control and Recovery in Distributed Databases;
  - 4.4. Overview of Transaction Management in Distributed Databases;
  - 4.5. Query Processing and Optimization in Distributed databases;
  - 4.6. Types of Distributed Database Systems;
  - 4.7. Distributed Database Architectures;
  - 4.8. Distributed Catalog Management.

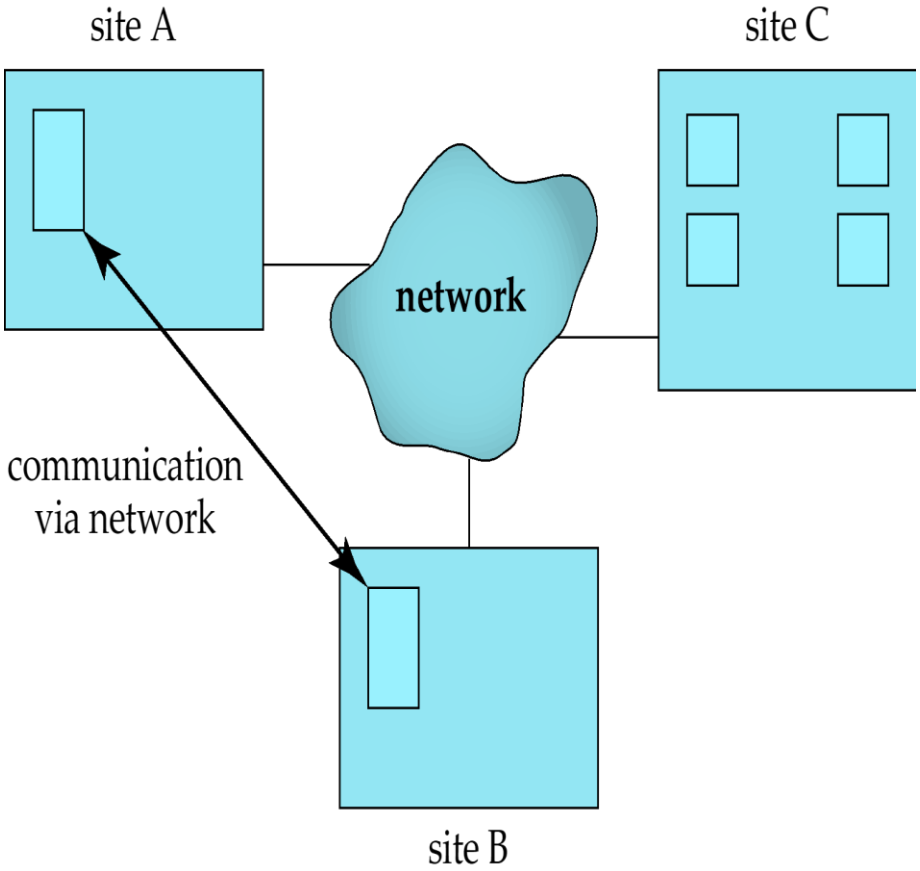
4.1. Distributed Databases (DDB) Concepts

4.1.1. Introduction to Distributed Database (DDB):



Cont'd…

- **A Distributed computing system:**
  - Consists of several processing sites or nodes interconnected by a computer network
  - Nodes cooperate in performing certain tasks.
  - Partitions large task into smaller tasks for solving efficiently in coordinated manner.



Cont'd…

- **Distributed Database (DDB):**
  - A collection of multiple logically interrelated databases distributed over a computer network.
  - Characterized by:
    - Connection of database nodes over computer network.
    - Logical interrelation of the connected databases.
    - Possible absence of homogeneity among connected nodes in terms of data, hardware and software.
  - A distributed database management system (DDBMS) is a software system that manages a distributed database.

Cont'd…

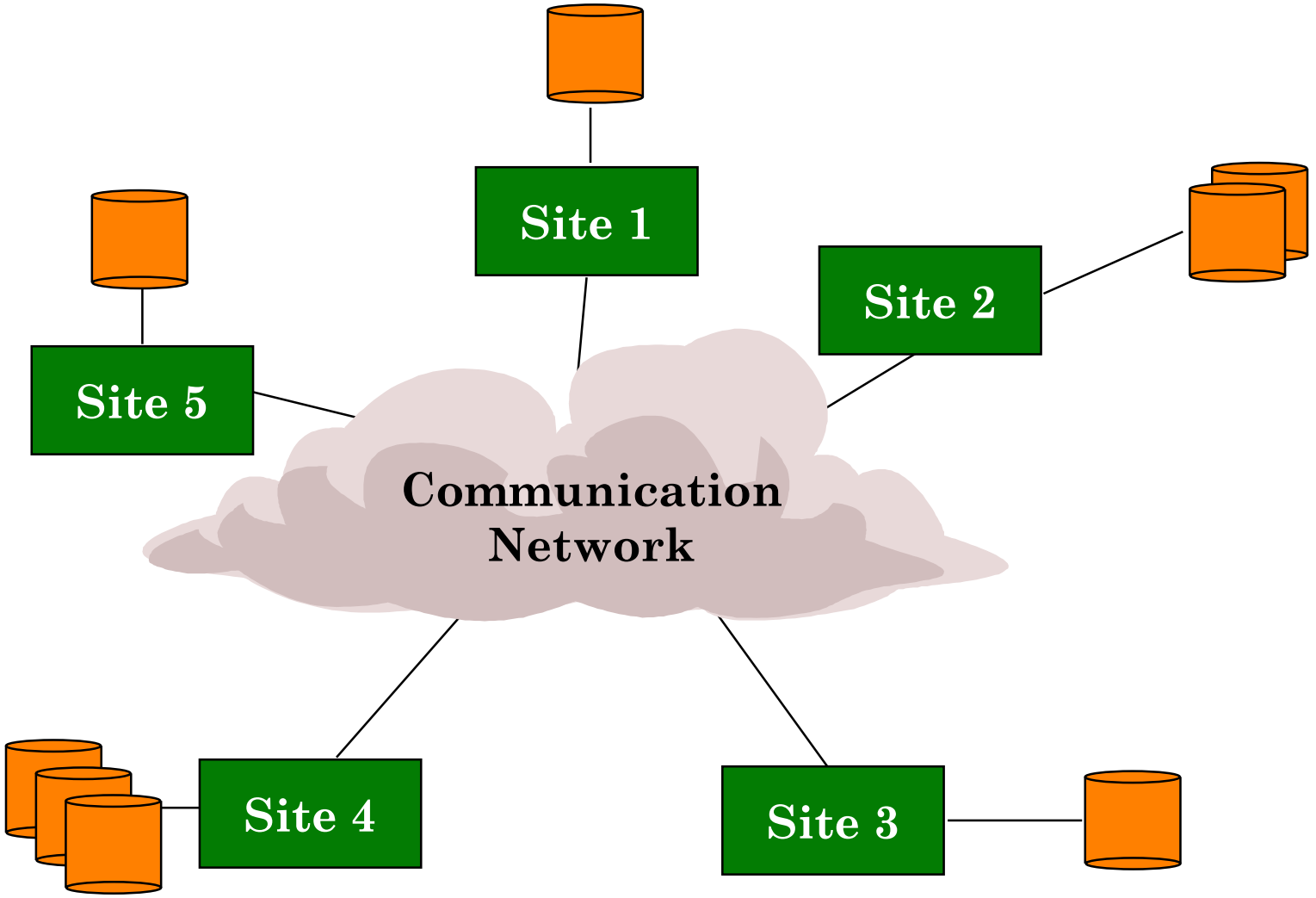


Figure: Distributed Database (DDB)

Cont'd…

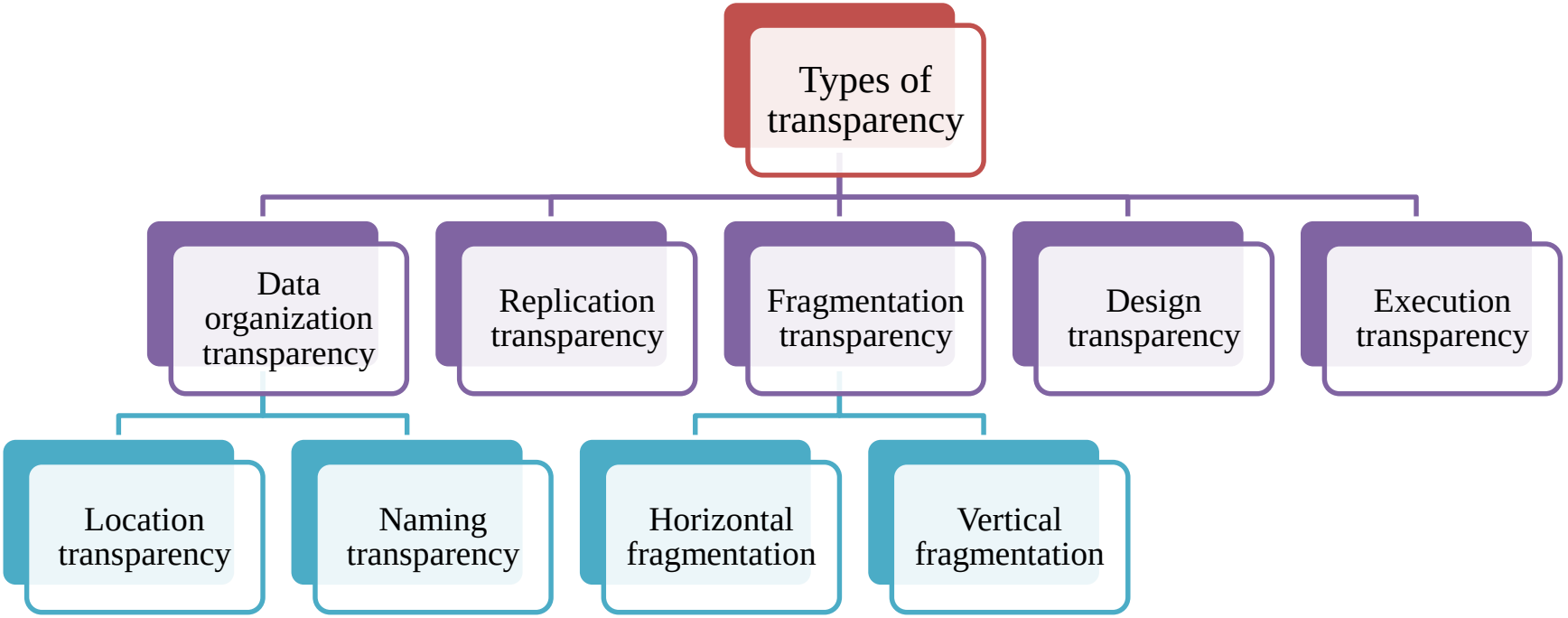
- **Advantages of Distributed Databases:**
  - Improved ease and flexibility of application development
    - Development at geographically dispersed sites
  - Increased availability
    - Isolate faults to their site of origin
  - Improved performance
    - Data localization
  - Easier expansion via scalability
    - Easier than in non-distributed systems

Cont'd…

- **4.1.2. Transparency:**
  - Hiding implementation details of the database systems from the end user while using the system.
  - This means users do not need to know:
    - Where the data is stored?
    - How the data is retrieved?
    - How the system is organized internally?
    - What hardware or software mechanisms are being used?
  - The system handles these details automatically and allow the user to use system with little or no awareness of underlying details.

Cont'd…

- **Types of Transparency:**



Cont'd…

- **Types of Transparency: 1. Data Organization Transparency**
  - Also known as distribution or network transparency.
  - Refers to freedom for the user to use distributed database systems without knowing the operational details of the network and the placement of the data in the distributed system.
  - **Two types:**
    - **Location Transparency**: user command used to perform a task is independent of the location of the data and the location of the node where the command was issued.
    - **Naming Transparency**: Once an object (relation, view, procedure, etc.) has a name, it can be accessed uniquely without specifying its location. The DBMS automatically determines the location.

Cont'd…

- **Types of Transparency: 2. Replication Transparency**
  - Copies of the same data objects may be stored at multiple sites for better availability, performance, and reliability.
  - Replication transparency makes the user unaware of the existence of these copies.
  - i.e., user issues the command to access the data object but does not know
    - How many copies exist?
    - Which copy is being accessed?
  - The DBMS automatically chooses the appropriate replica.

Cont'd…

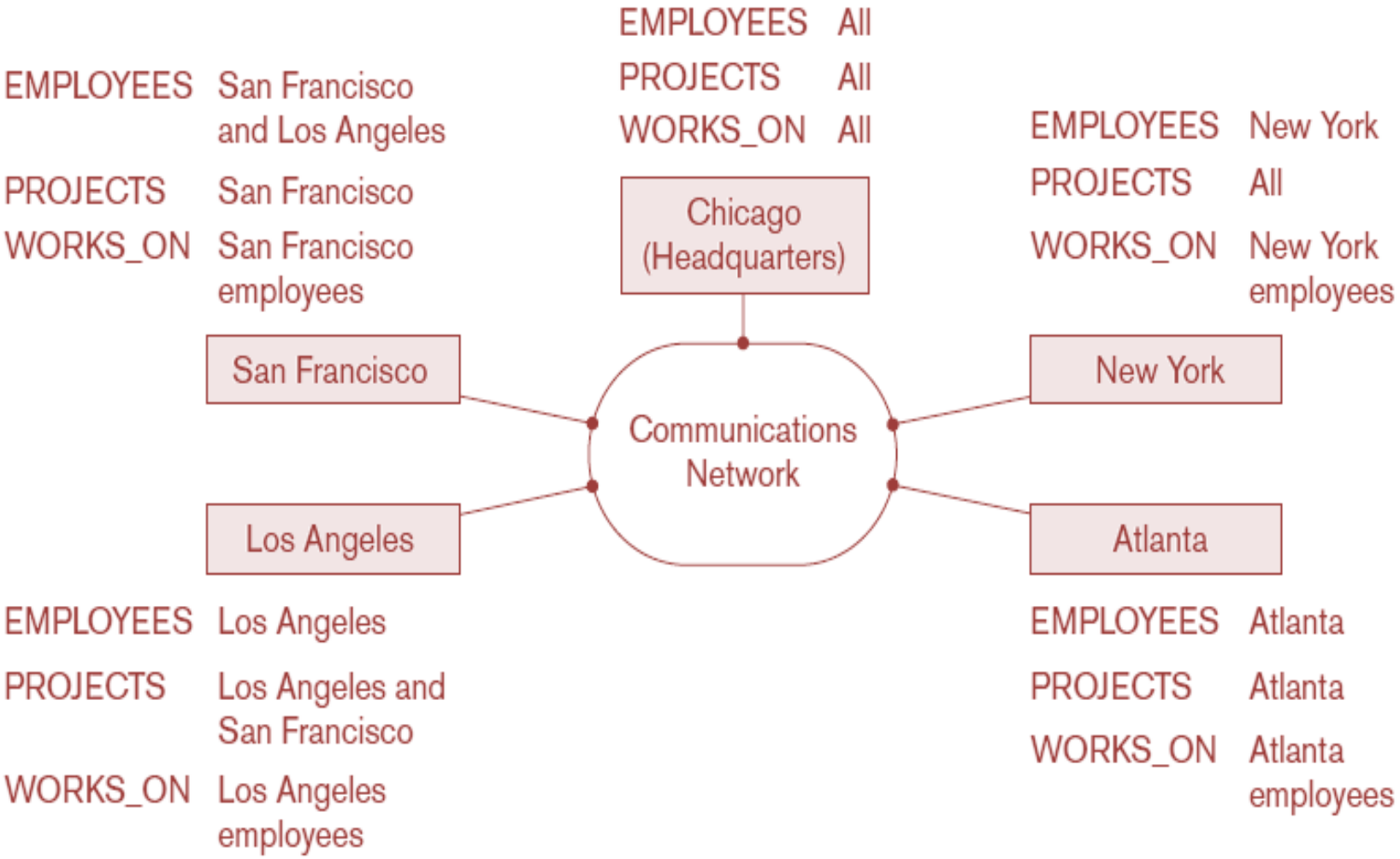


Figure: Data distribution and replication among distributed databases

Cont'd…

- **Types of Transparency: 3. Fragmentation Transparency**
  - A table may be divided into smaller pieces called fragments.
  - Users see the original table even though it is physically divided.
  - Two types of fragmentation are possible.
    - **Horizontal fragmentation (sharding)** distributes a relation (table) into sub-relations that are subsets of the tuples (rows) in the original relation;
    - **Vertical fragmentation** distributes a relation into sub-relations where each sub-relation is defined by a subset of the columns of the original relation.
  - Fragmentation transparency makes the user unaware of the existence of fragments.

Cont'd…

- **Types of Transparency: 4. Design Transparency**
  - Design transparency makes the user unaware of :
    - How the database is fragmented
    - Where fragments are stored
    - How replicas are maintained
    - How the distributed database was designed
  - The design details are hidden by the DDBMS from the user.

Cont'd…

- **Types of Transparency: 5. Execution Transparency**
  - Execution transparency makes the user unaware of:
    - Where a transaction executes
    - Which site processes the query
    - How data is transferred between sites

Cont'd…

- **4.1.3. Availability and Reliability**:
  - Availability
    - Probability that the system is continuously available during a time interval
    - Focuses on continuous service.
  - Reliability
    - Probability that the system is running (not down) at a certain time point.
    - Focuses on correctness at a particular moment.
  - Both depends on handling faults, errors, and failures.
  - A DDBMS improves these through fault tolerance, replication, recovery mechanisms, and the ability to continue operation despite transaction, hardware, or network failures.

Cont'd…

- **4.1.4. Scalability and Partition Tolerance**:
  - The extent to which the system can expand its capacity while continuing to operate without interruption.
  - There are two types of scalability:
    - Horizontal scalability: Expanding the number of nodes in a distributed system.
    - Vertical scalability: Expanding capacity of the individual nodes.
  - **Partition tolerance** is the ability of a distributed system to continue operating even when network failures split the system into multiple disconnected groups.

Cont'd…

- **4.1.5. Autonomy**
  - Extent to which individual nodes can operate independently.
  - A high degree of autonomy is desirable for increased flexibility and customized maintenance of an individual node.
  - Autonomy can be:
    - Design autonomy: Independence of data model usage and transaction management techniques among nodes
    - Communication autonomy: Determines the extent to which each node can decide on sharing information with other nodes
    - Execution autonomy: Independence of users to act as they please

4.2 Data Fragmentation, Replication, and Allocation Techniques for Distributed Database Design

- In distributed database design process following techniques are used to allow certain data to be stored in more than one site to increase availability and reliability:
  - **Data Fragmentation**: Break up the database into logical units called fragments that are to be distributed.
  - **Data Replication**: Store copy of same data (replica) in more than one site.
  - **Data Allocation**: decide on how to distribute the data among nodes.

Cont'd…

- **4.2.1. Data Fragmentation and Sharding:**
  - Data fragmentation refers to splitting a relation into logically related and correct parts (Fragments).
  - A relation can be fragmented in three ways:
    - Horizontal Fragmentation
    - Vertical Fragmentation
    - Hybrid Fragmentation

Cont'd…

- **Horizontal fragmentation (sharding):**
  - Horizontal fragment or shard of a relation is a subset of the tuples in that relation
  - Can be specified by condition on one or more attributes: σ_{Ci} (R)
  - Groups rows to create subsets of tuples where, each subset has a certain logical meaning
  - Consider the Employee relation with selection condition (DNO = 5). All tuples satisfy this condition will create a subset which will be a horizontal fragment of Employee relation.
  - To reconstruct R from horizontal fragments a UNION is applied.

Cont'd…

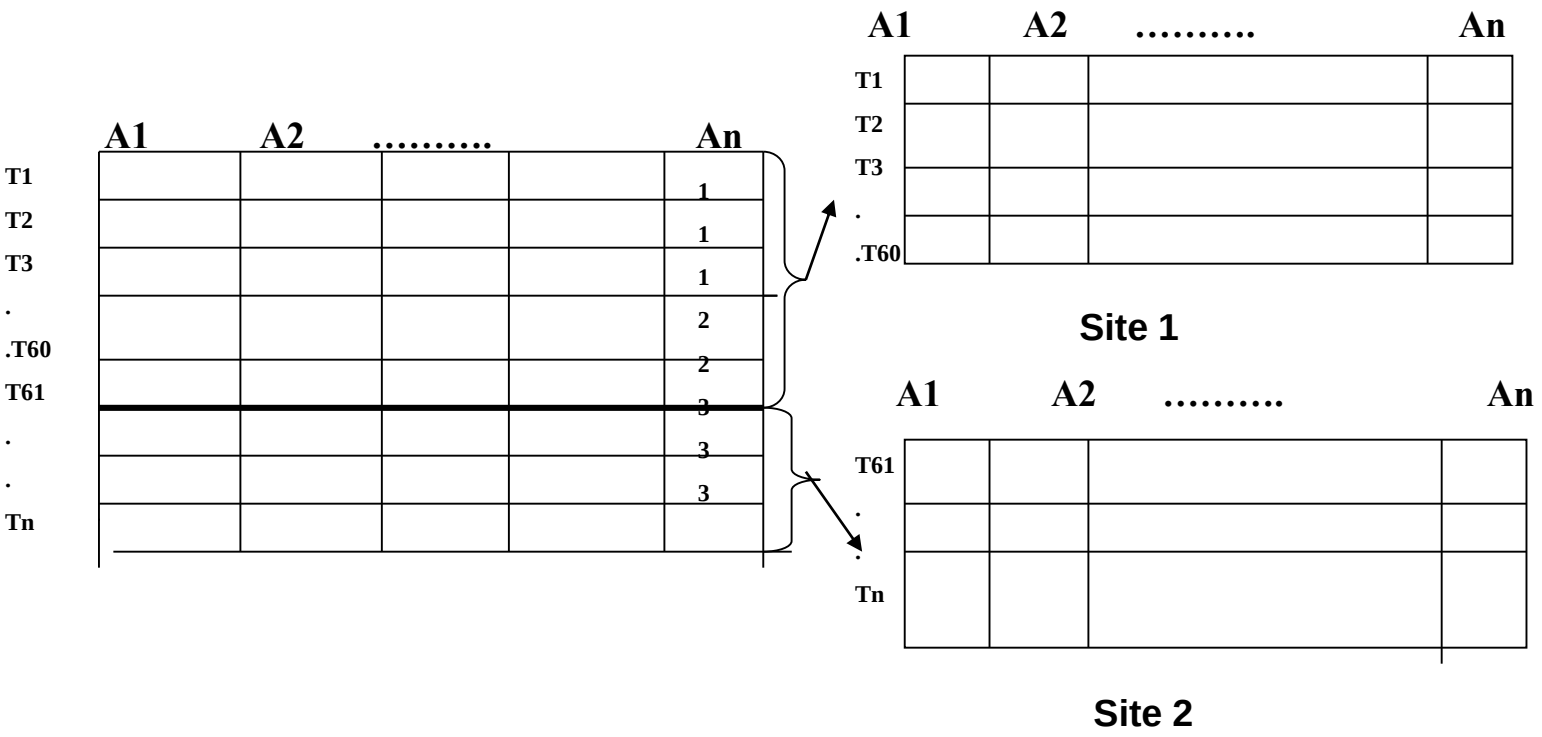
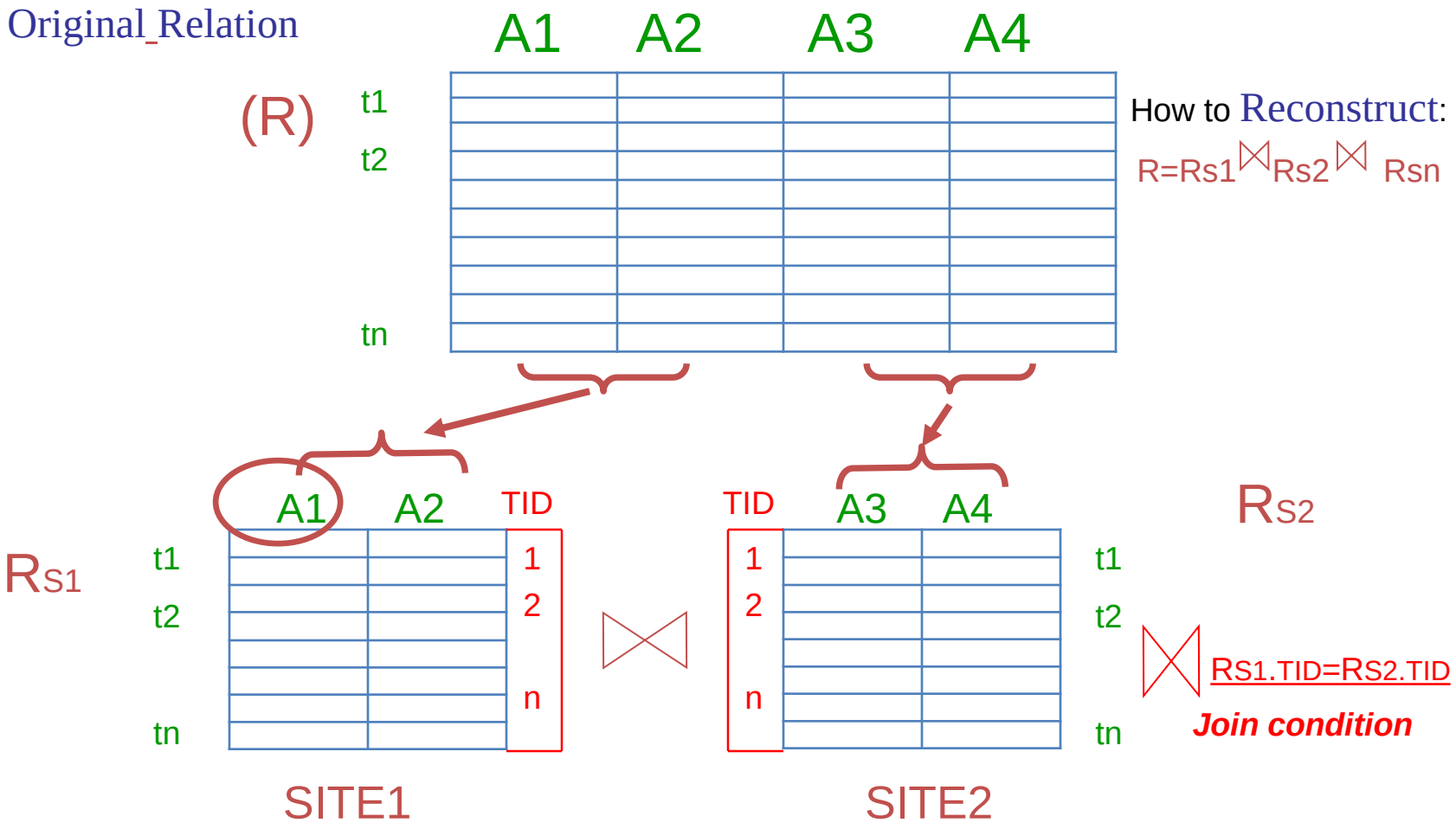


Figure: Horizontal fragmentation (sharding)

Cont'd…

- **Vertical fragmentation:**
  - Divides a relation vertically by columns
  - Can be specified by a $\Pi_{Li}(R)$
  - Keeps only certain attributes of the relation
  - Each fragment must include the primary key attribute of the parent relation.
  - Consider the Employee relation. A vertical fragment of can be created by keeping the values of Name, Bdate, Sex, and Address.
  - To reconstruct R from complete vertical fragments a OUTER UNION is applied.

Cont'd…

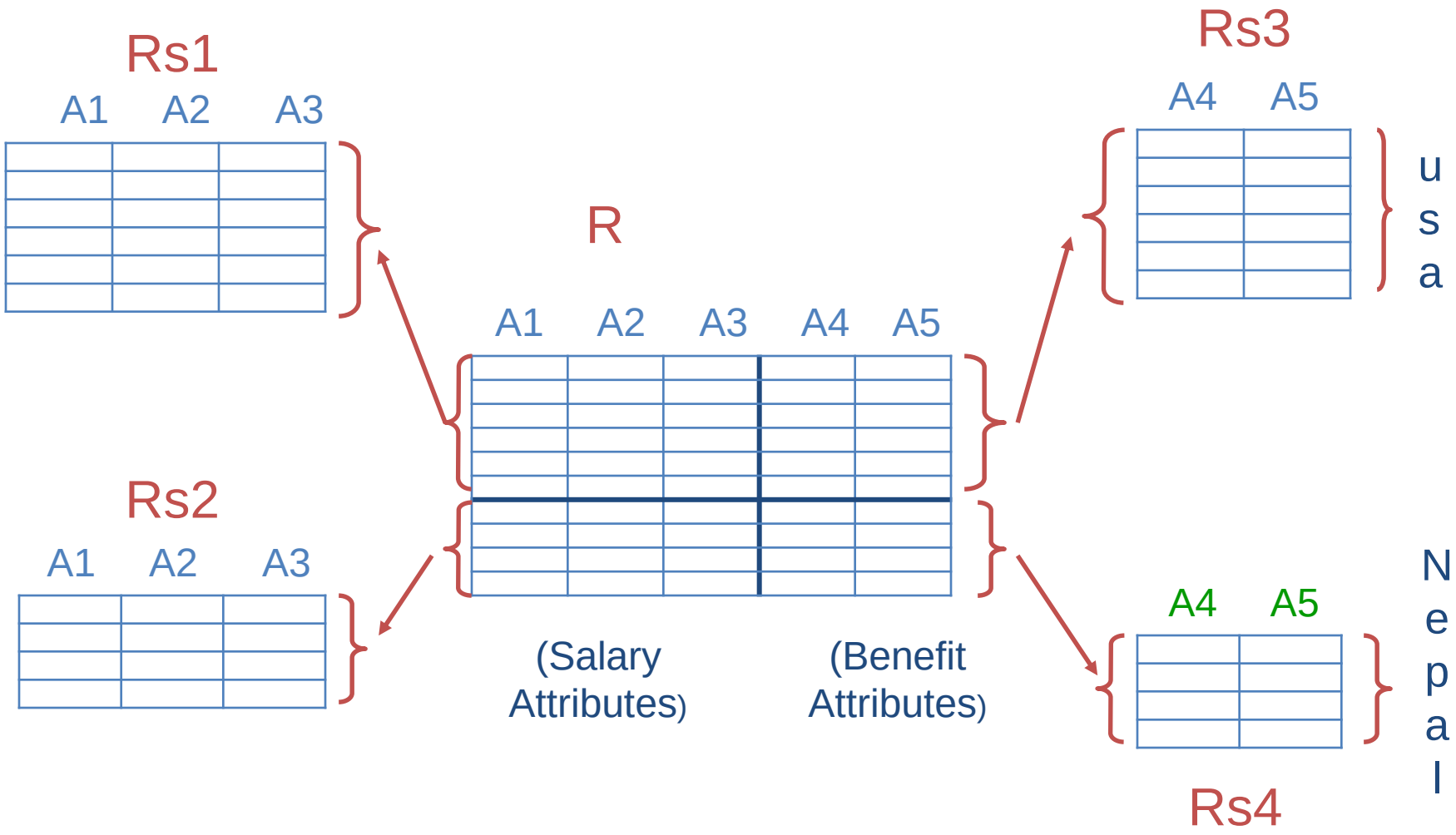


TID –Tuple ID Hidden Attribute to ensure account and simple join reconstruction

Cont'd…

- **Mixed (hybrid) fragmentation:**
  - Combination of horizontal and vertical fragmentations
  - This is achieved by SELECT-PROJECT operations which is represented by $\Pi_{Li}(\sigma_{Ci}(R))$.
  - If C = True (Select all tuples) and L partially includes ATTRS(R), we get a vertical fragment, and
  - If C is partially True and L also partially includes ATTRS(R), we get a mixed fragment.
  - If C = True and L = ATTRS(R), then R can be considered a fragment.

Cont'd…


Rs1
A1
A2
A3
Rs2
A1
A2
A3
R
A1
A2
A3
A4
A5
(Salary Attributes)
(Benefit Attributes)
Rs3
A4
A5
usa
Nepal
A4
A5
Rs4

Cont'd…

- **4.2.2. Data Replication**:
  - To improve the availability of the data DDB allow to store same fragment of the data at more than one site called the replication of the data.
  - Three types of Data replication:
    - Fully replicated distributed database
      - Replication of whole database at every site in distributed system
      - Improves availability remarkably
      - Update operations can be slow
    - Nonredundant allocation (no replication)
      - Each fragment is stored at exactly one site
    - Partial replication
      - Some fragments are replicated and others are not
      - Defined by replication schema

Cont'd…

- **4.2.3. Data allocation (data distribution):**
  - Data distribution is the process of assigning each fragment (or its copies) to specific sites in a distributed database system.
  - It determines where data is stored across different networked sites.
  - Choices depend on performance and availability goals of the system and on the types and frequencies of transactions submitted at each site.
  - For example,
    - If high availability is required and most transactions are read-only, a fully replicated database is suitable.
    - If certain data is mostly accessed at a specific site, its fragments should be allocated locally at that site.
    - Data accessed at multiple sites can be replicated across those sites.
    - If there are many update operations, replication should be limited to reduce overhead.

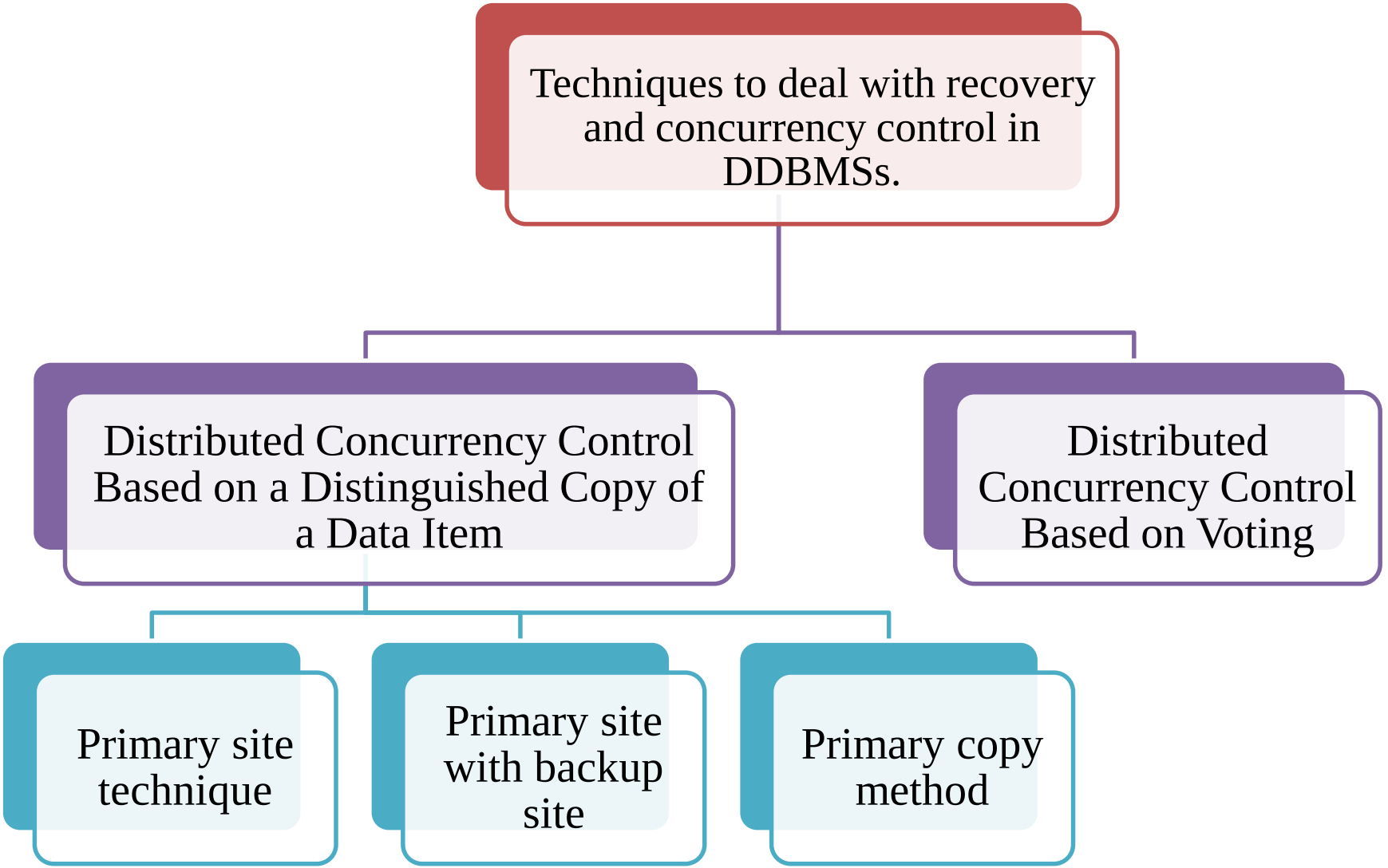
Cont'd…

- The information concerning data fragmentation, allocation, and replication is stored in a global directory that is accessed by the DDBS applications as needed.
  - Fragmentation schema:
    - Defines a set of fragments that includes all attributes and tuples in the database
  - Allocation schema:
    - Describes the allocation of fragments to nodes of the DDBS

4.3 Overview of Concurrency Control and Recovery in Distributed Databases

- In distributed database environment, following distributed database concurrency control and recovery specific problem may occurs:
  - Multiple copies of the data items
  - Failure of individual sites
  - Failure of communication links
  - Distributed commit
  - Distributed deadlock
- Distributed concurrency control and recovery techniques must deal with these problems.

Cont'd…


Techniques to deal with recovery and concurrency control in DDBMSs.
Distributed Concurrency Control Based on a Distinguished Copy of a Data Item
Distributed Concurrency Control Based on Voting
Primary site technique
Primary site with backup site
Primary copy method

Cont'd…

- **4.3.1. Distributed Concurrency Control Based on a Distinguished Copy of a Data Item**:
  - A particular copy of each data item is designated as a **distinguished copy**.
  - The locks for this data item are associated with the distinguished copy.
  - All locking and unlocking requests are sent to the site containing Distinguished copy.
  - A site that includes a distinguished copy of a data item basically acts as the coordinator site for concurrency control on that item.
  - Based on method of choosing the distinguished copies it has three types:
    - Primary site technique
    - Primary site with backup site
    - Primary copy method

Cont'd…

- **Primary site technique:**
  - All distinguished copies kept at the same site called a primary site is designated to be the coordinator site for all database items.
  - Hence, all locks are kept at that site, and all requests for locking or unlocking are sent there.
  - Once a lock is granted, the data items themselves can be accessed at any site at which they reside.
  - DDBMS is responsible for updating all copies of the data item before releasing the lock.
  - In Case of coordinator fails, new coordinator is elected based on the majority voting (yes) by aborting the currently running transactions.
  - **Advantages**: Simple, **Disadvantages**: system bottleneck, single point of failure hence limits the reliability and availability.

Cont'd…

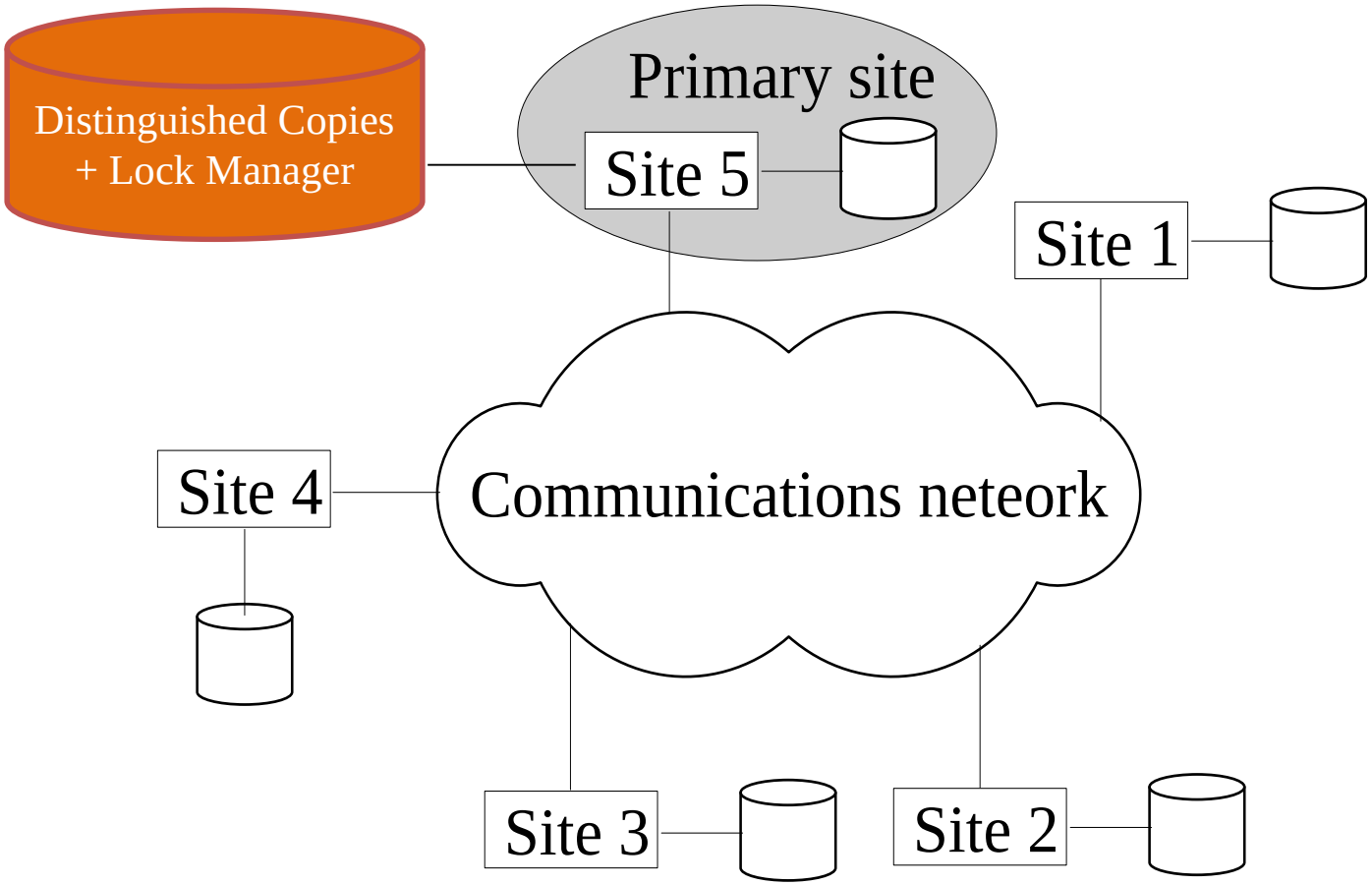


Figure: Primary site technique

Cont'd…

- **Primary site with backup site:**
  - All locking information is maintained at both the primary site and the backup site.
  - In case of primary site failure (coordinator), the backup site takes over as the primary site, and a new backup site is chosen.
  - **Advantages**: Eliminates single point of failure
  - **Disadvantage**: slows down the process of acquiring locks in case of primary site failure.

Cont'd…

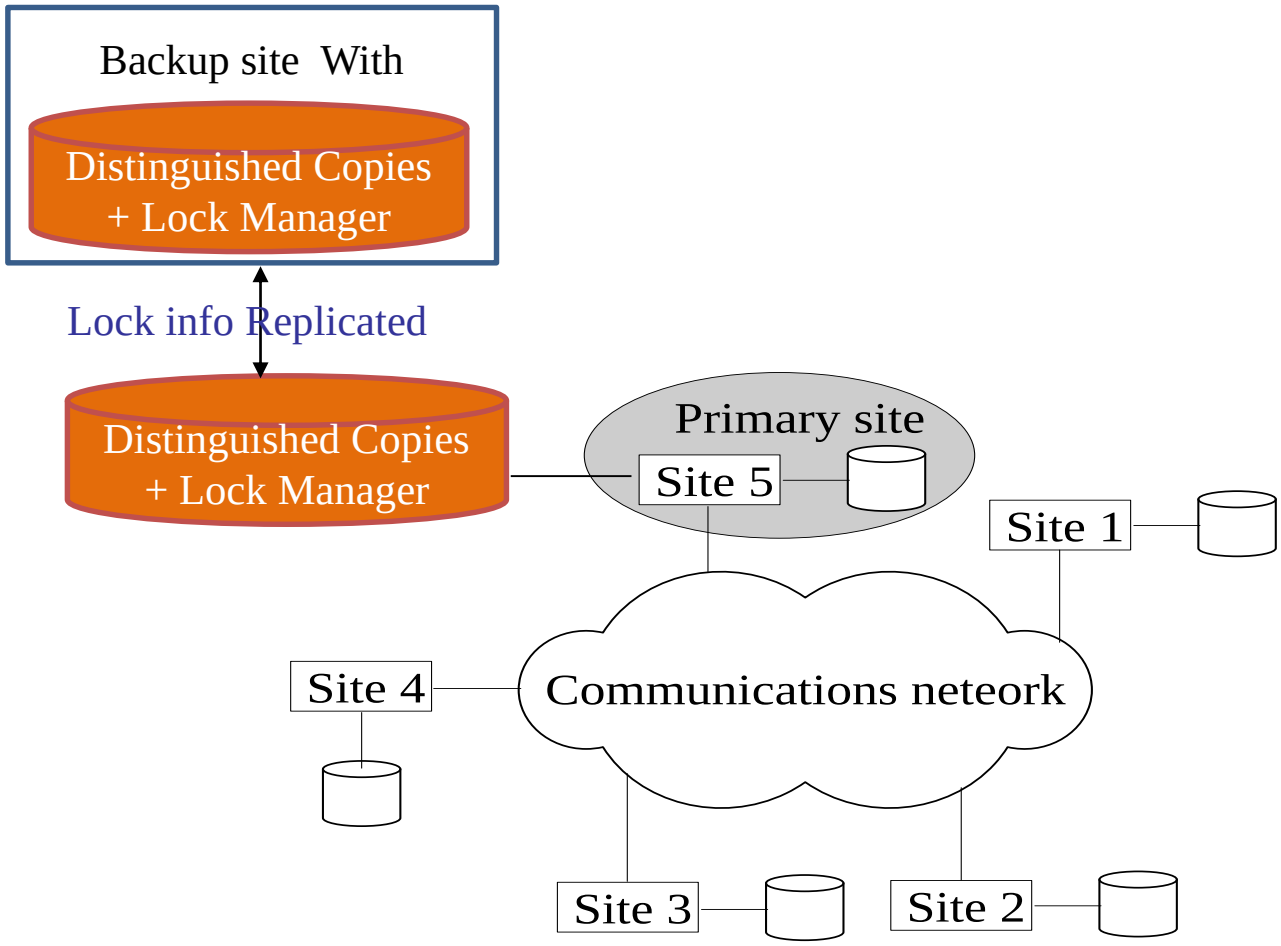


Figure: Primary site with Backup site

Cont'd…

- **Primary copy method**:
  - Distribute the load of lock coordination among various sites by having the distinguished copies of different data items stored at different sites.
  - Failure of one site affects any transactions that are accessing locks on items whose primary copies reside at that site, but other transactions are not affected.
  - This method can also use backup sites to enhance reliability and availability.

Cont'd…

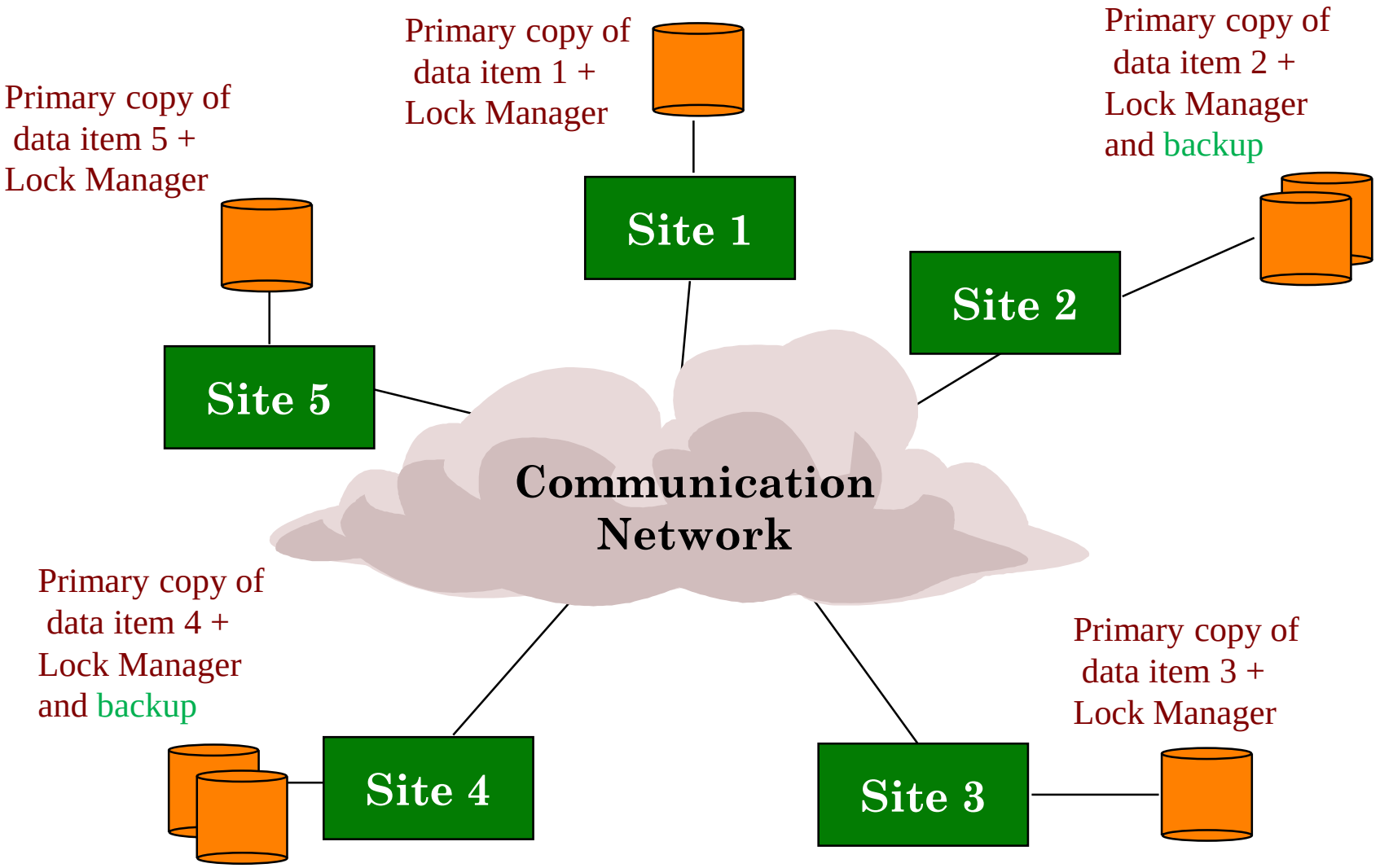
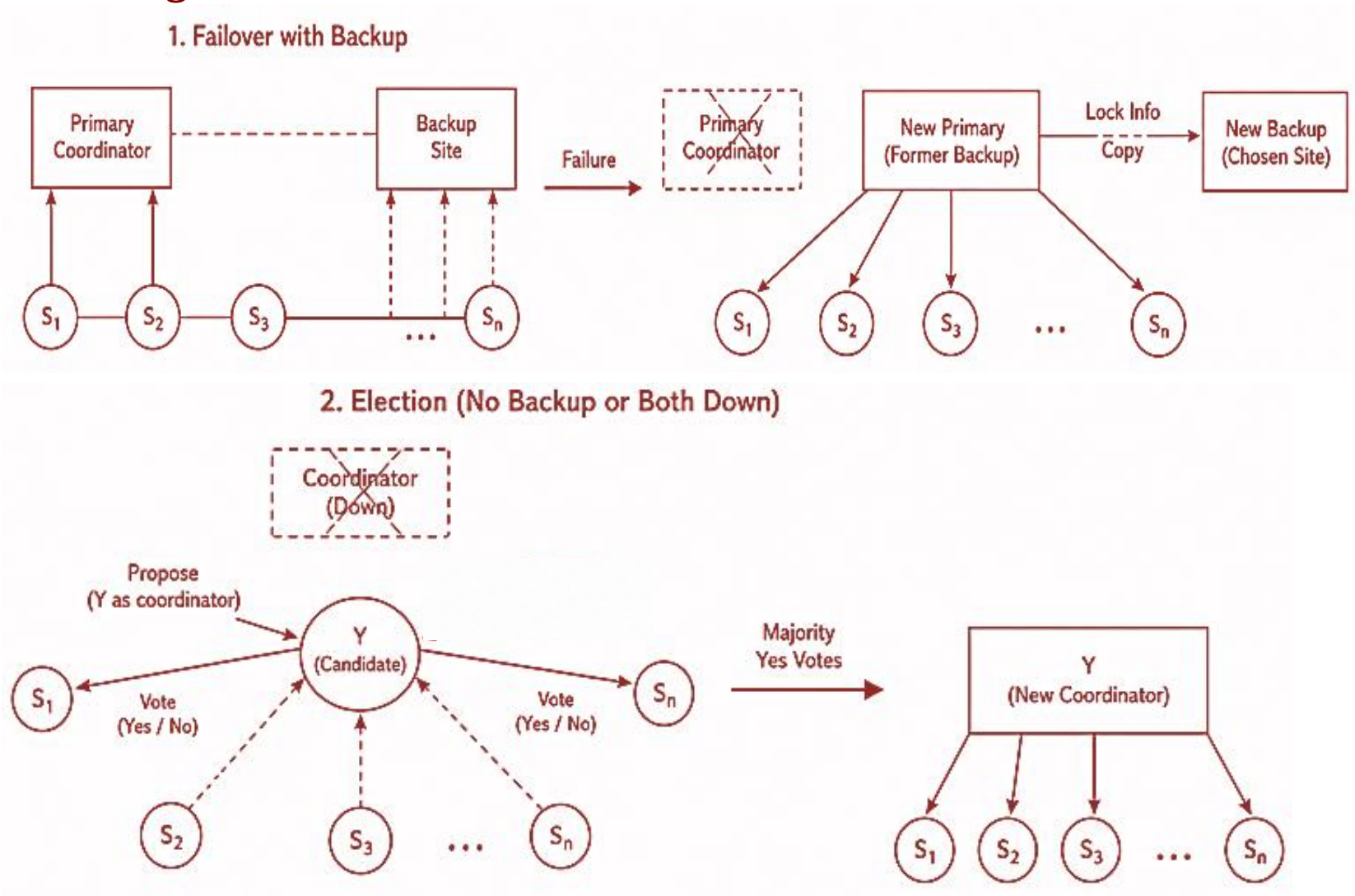


Figure: Distributed Database (DDB)

Cont'd…

- **Choosing a New Coordinator Site in Case of Failure**:



Cont'd…

- **4.3.2. Distributed Concurrency Control Based on Voting:**
  - In this method, there is no distinguished copy; rather, a lock request is sent to all sites that includes a copy of the data item.
  - Each copy maintains its own lock and can grant or deny the request for it.
  - If a transaction that requests a lock is granted that lock by a majority of the copies, it holds the lock and informs all copies that it has been granted the lock.
  - If a transaction does not receive a majority of votes granting it a lock within a certain time-out period, it cancels its request and informs all sites of the cancellation.
  - Truly distributed control, as there is no any single coordinator.
  - Results in higher message traffic among sites

Cont'd…

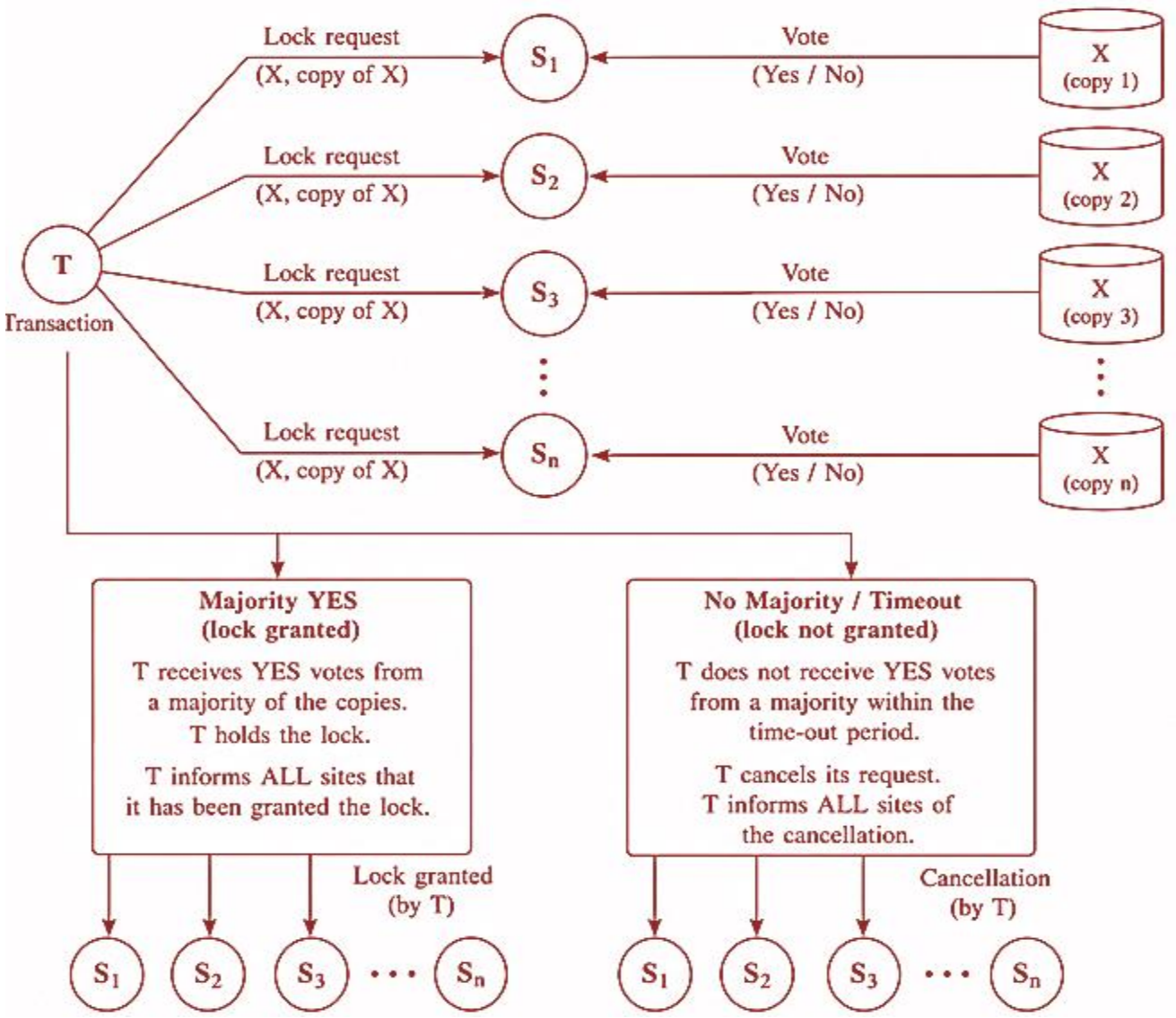


Figure: Distributed Concurrency Control Based on Voting:

Cont'd…

- **4.3.1. Distributed Recovery**:
  - The recovery process in distributed databases is quite involved.
  - Some of the issues associated with distributed recovery includes the following:
    - Difficult to determine whether a site is down without exchanging numerous messages with other sites
    - Distributed commit: When a transaction is updating data at several sties, it cannot commit until certain its effect on every site cannot be lost
      - Two-phase commit protocol often used to ensure correctness

4.4 Overview of Transaction Management in Distributed Databases

- TM – Transaction Manager, SC- Schedular, RM – Recovery Manager.

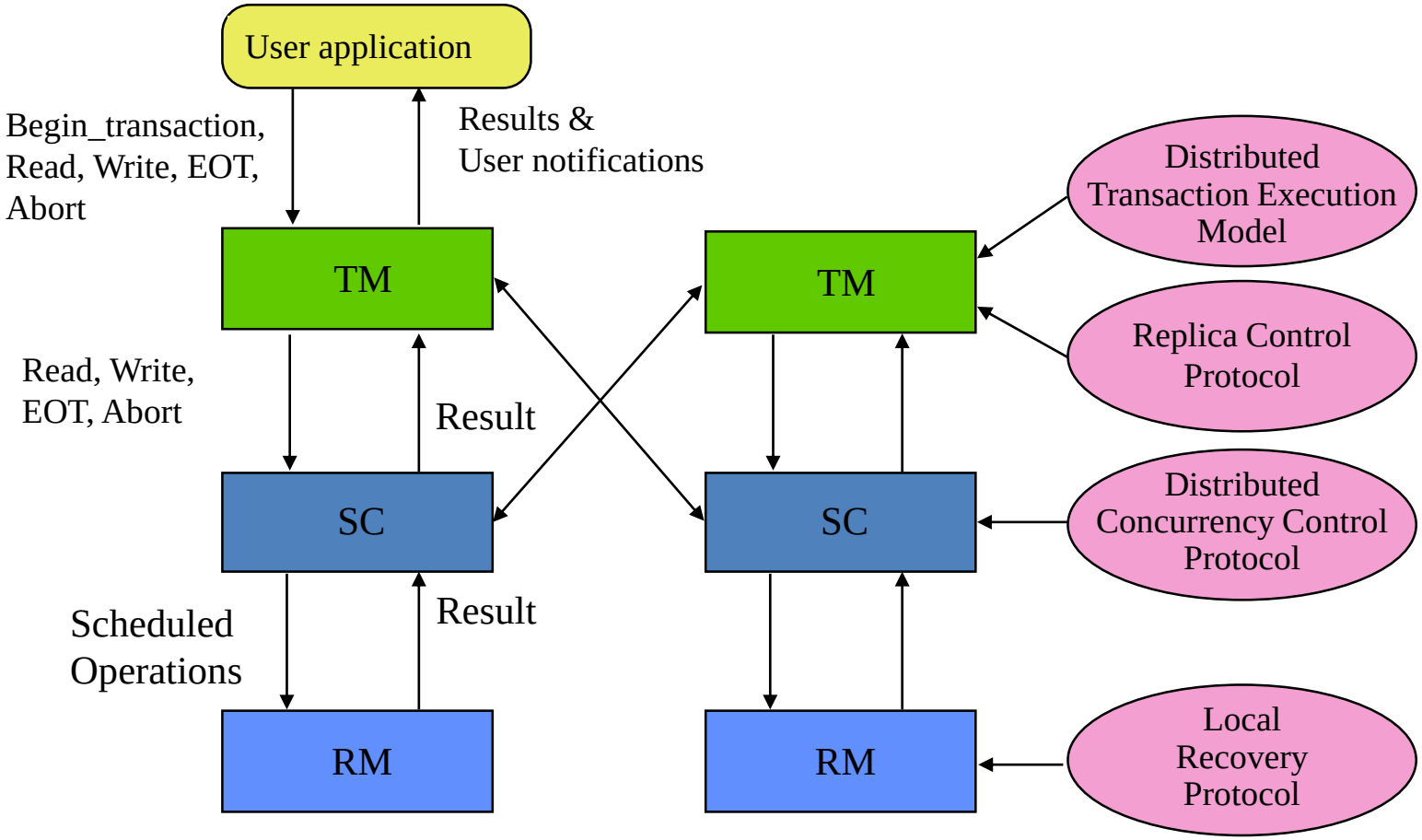


Figure: Distributed Transaction Execution

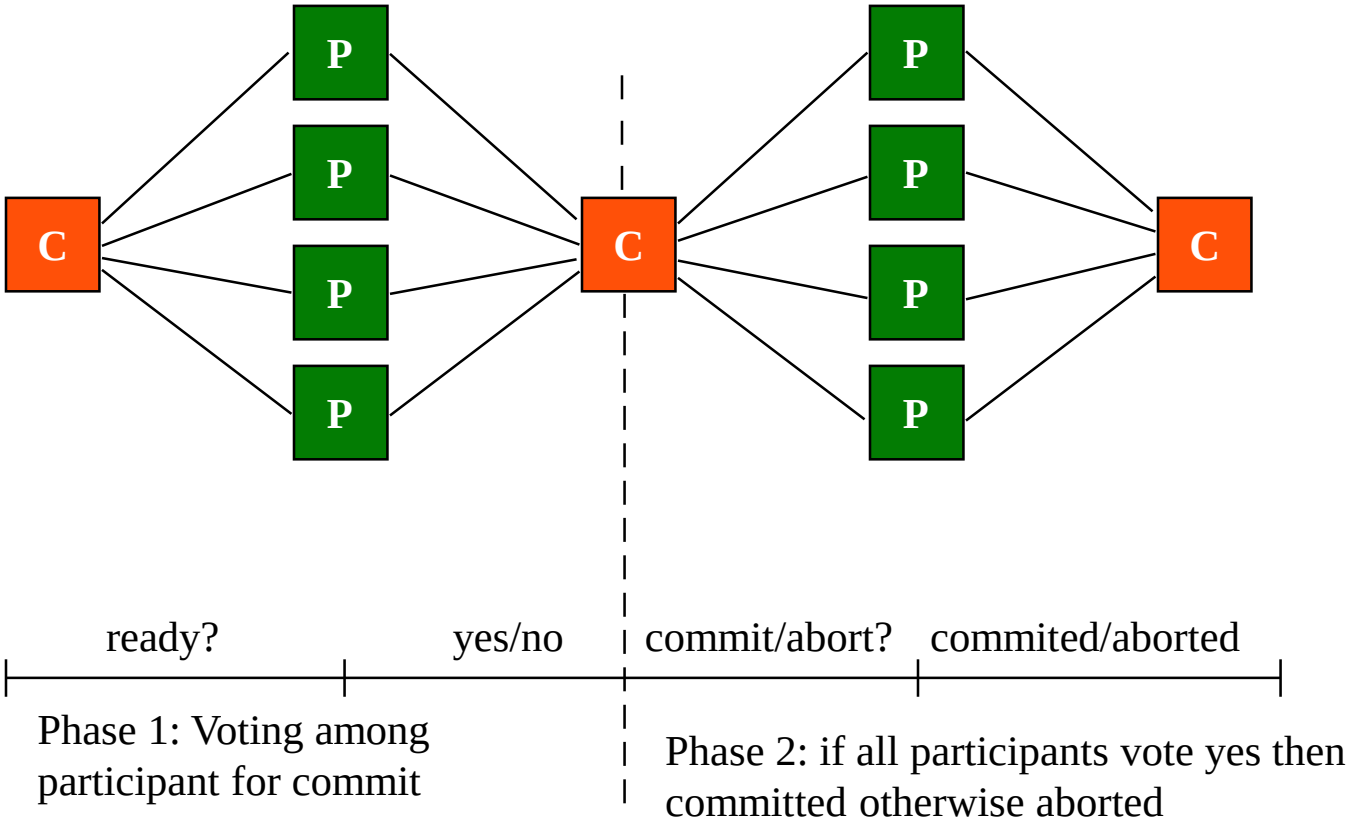
The global and local transaction manager, along with the concurrency control and recovery manager of a DDBMS, collectively guarantee the ACID properties of transactions.

Cont'd…

- Global transaction manager (coordinator): is transaction manager at the site where the transaction originates. It coordinates the execution of the transaction with the transaction managers at multiple sites.
- Transaction Managers passes database operations and associated information to the concurrency controller for acquisition and release of locks.
  - **Operations**: BEGIN_TRANSACTION, READ or WRITE, END_TRANSACTION, COMMIT_TRANSACTION, and ROLLBACK or ABORT
  - **Information's**: unique identifier, originating site, name, and so on.
- If the transaction requires access to a locked resource, it is blocked until the lock is acquired.
- Once the lock is acquired, the operation is sent to the runtime processor, which handles the actual execution of the database operation.
- Once the operation is completed, locks are released and the transaction manager is updated with the result of the operation.

Cont'd…

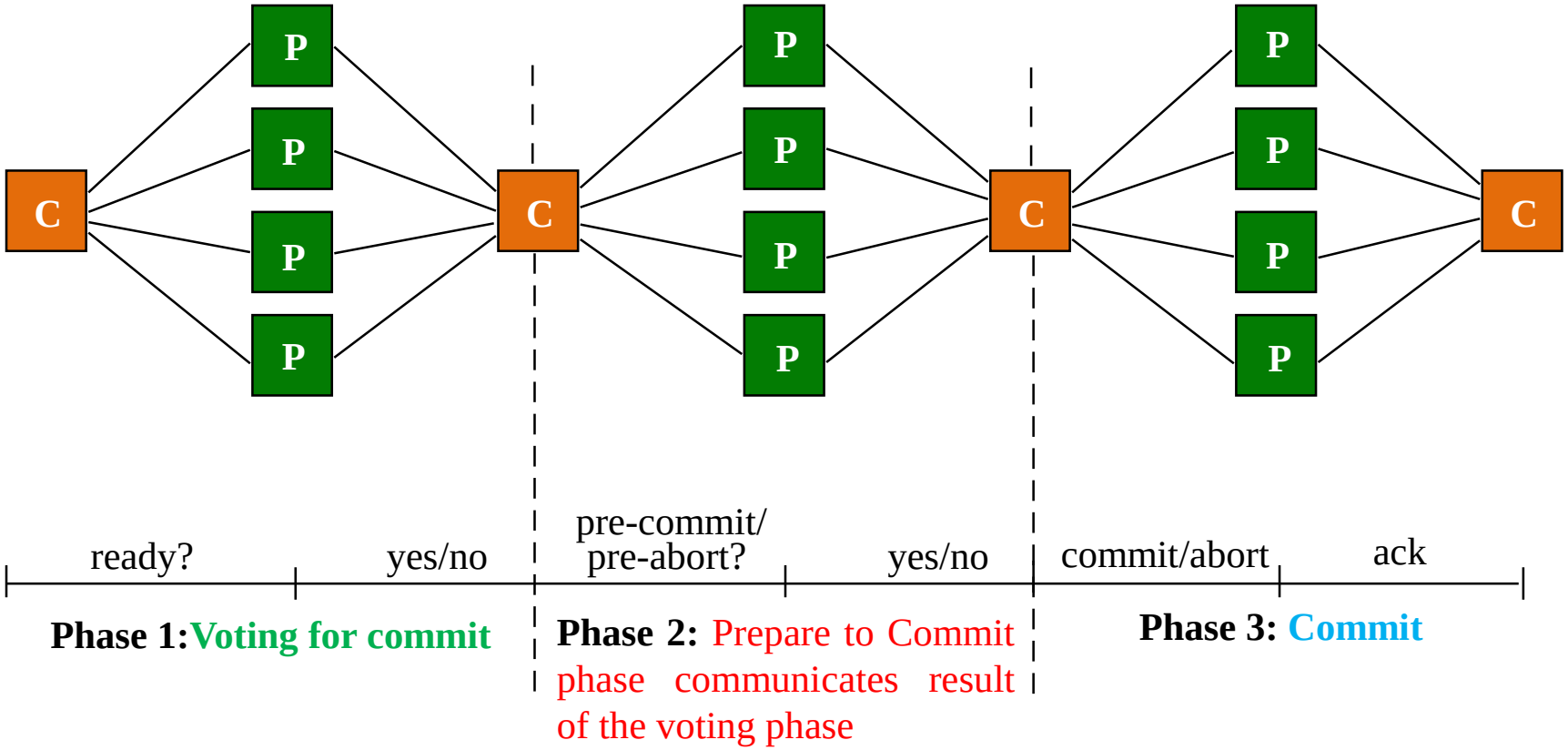
- **Commit Protocols:** Two-phase



C- Coordinator or Global Transaction Manager, P- Participants or other TM

Cont'd…

- **Commit Protocols:** Three-phase divides second commit phase into two subphases.



4.5 Query Processing and Optimization in Distributed Databases

- **4.5.1.Distributed Query Processing:** Stages of a distributed database query processing includes the following:
  - Query mapping: As in centralized DB, query is parsed and translated into relational algebraic expressions on global relations refers to global conceptual schema.
    - Do not consider actual distribution and replication of data.
  - Localization: Maps the distributed query on global schema to separate queries on individual fragments using data distribution and replication information.
  - Global query optimization: Near optimal strategy is selected from list of candidates strategy to execute each component queries. Heuristics like reordering the operations and the network cost is the major factor for optimization.
  - Local query optimization: This stage is common to all sites in the DDB. The techniques are similar to those used in centralized systems.

Cont'd…

- **4.5.2. Data Transfer Costs of Distributed Query Processing**:
  - Major cost in distribute query processing is the cost of transferring intermediate and final result files through the computer network.
  - This cost is usually high so some optimization is necessary.
  - **Optimization criterion:** Reducing amount of data transfer

Cont'd…

- **Example relations**: **Employee** at site 1 and **Department** at Site 2
  - Employee at site 1 has 10,000 rows. Row size = 100 bytes. Table size = 10^6 bytes.

Fname	Minit	Lname	SSN	Bdate	Address	Sex	Salary	Superssn	Dno

  - Department at Site 2 has100 rows. Row size = 35 bytes. Table size = 3,500 bytes.

Dname	Dnumber	Mgrssn	Mgrstartdate

    - Q: For each employee, retrieve employee name and department name Where the employee works.
    - Q: $\Pi_{Fname,Lname,Dname}$ (Employee$\bowtie_{Dno = Dnumber}$ Department)

Cont'd…

- **Result**
  - The result of this query will have 10,000 tuples, assuming that every employee is related to a department.
  - Suppose each result tuple is 40 bytes long. The query is submitted at site 3 and the result is sent to this site.
  - **Problem:** Employee and Department relations are not present at site 3.

Cont'd…

- **Strategies:**
  1. Transfer Employee and Department to site 3.
     - Total transfer bytes = 1,000,000 + 3500 = 1,003,500 bytes.
  2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3.
     - Query result size = 40 * 10,000 = 400,000 bytes.
     - Total transfer size = 400,000 + 1,000,000 = 1,400,000 bytes.
  3. Transfer Department relation to site 1, execute the join at site 1, and send the result to site 3.
     - Total bytes transferred = 400,000 + 3500 = 403,500 bytes.
- **Optimization criteria**: minimizing data transfer.
- Preferred approach: strategy 3.

Cont'd…

- **Consider the query**
  - **Q'**: For each department, retrieve the department name and the name of the department manager
  - Relational Algebra expression:
    - $\Pi_{Fname,Lname,Dname}$ (Employee$\bowtie_{Mgrssn = SSN}$ Department)

Cont'd…

- The result of this query will have 100 tuples, assuming that every department has a manager, the execution strategies are:
  1. Transfer Employee and Department to the result site and perform the join at site 3.
     - Total bytes transferred = 1,000,000 + 3500 = 1,003,500 bytes.
  2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3.
     - Query result size = 40 * 100 = 4000 bytes.
     - Total transfer size = 4000 + 1,000,000 = 1,004,000 bytes.
  3. Transfer Department relation to site 1, execute join at site 1 and send the result to site 3.
     - Total transfer size = 4000 + 3500 = 7500 bytes.
- **Optimization criteria**: minimizing data transfer.
- Preferred strategy: Choose strategy 3.

Cont'd…

- **Now suppose the result site is 2. Possible strategies** :

  1. Transfer Employee relation to site 2, execute the query and present the result to the user at site 2.
     - Total transfer size = 1,000,000 bytes for both queries Q and Q'.
  2. Transfer Department relation to site 1, execute join at site 1 and send the result back to site 2.
     - Total transfer size for Q = 400,000 + 3500 = 403,500 bytes and for Q' = 4000 + 3500 = 7500 bytes.

- **Optimization criteria**: minimizing data transfer.
- Preferred strategy: Choose strategy 2.

Cont'd…

- **4.5.3. Distributed query processing using semi-join**
  - Reduces the number of tuples in a relation before transferring it to another site
  - Send the joining column of one relation R to one site where the other relation S is located
  - Join attributes and result attributes shipped back to original site
  - Efficient solution to minimizing data transfer

Cont'd…

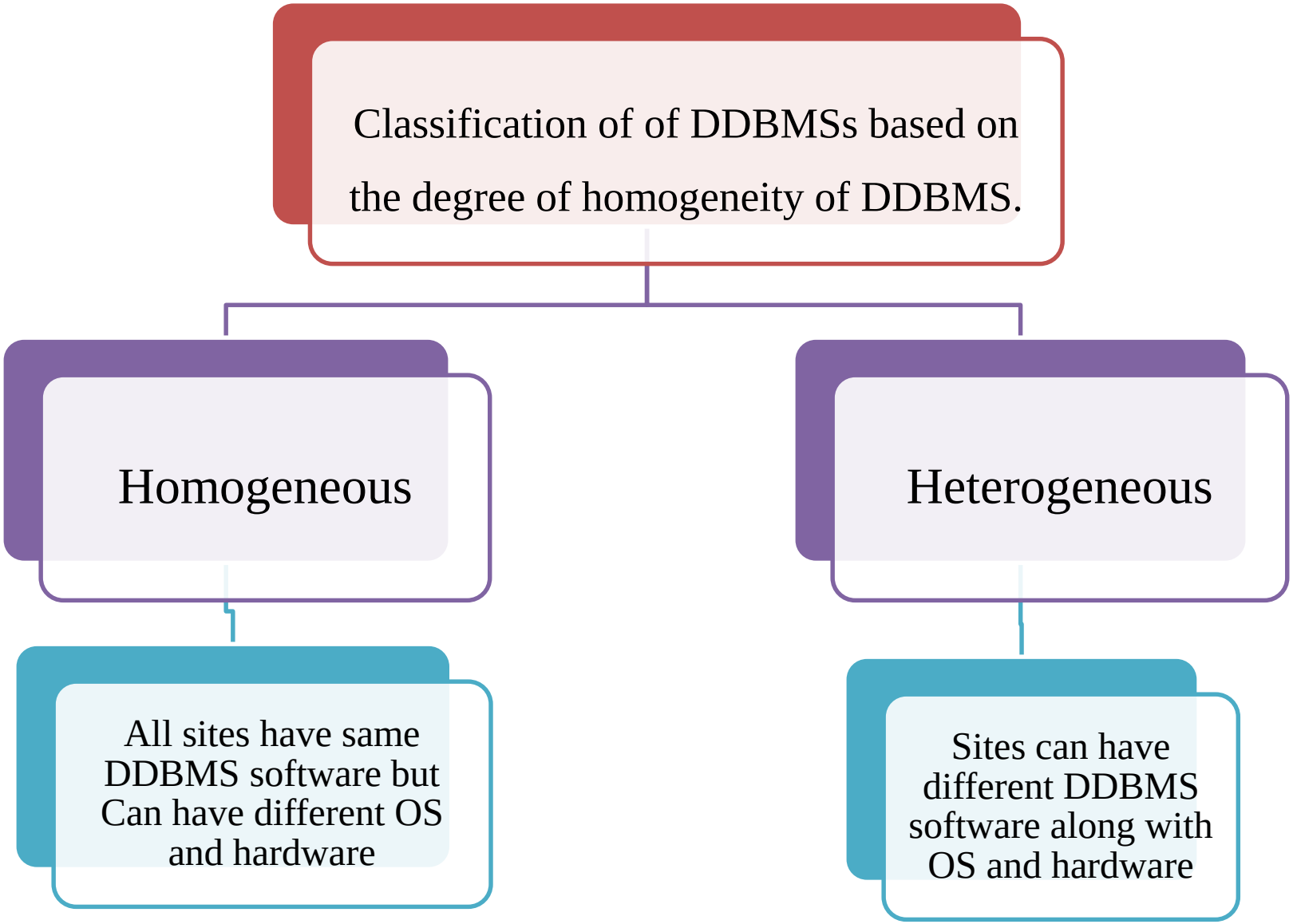
- **Example execution of Q or Q':**

1. Project the join attributes of Department at site 2, and transfer them to site 1.
   - For Q, 4 * 100 = 400 bytes are transferred and
   - For Q', 9 * 100 = 900 bytes are transferred.
2. Join the transferred file with the Employee relation at site 1, and transfer the required attributes from the resulting file to site 2.
   - For Q, 34 * 10,000 = 340,000 bytes are transferred and
   - For Q', 39 * 100 = 3900 bytes are transferred.
3. Execute the query by joining the transferred file with Department and present the result to the user at site 2.

Cont'd…

- **4.5.4. Query and update decomposition**
  - User can specify a query as if the DBMS were centralized
    - If full distribution, fragmentation, and replication transparency are supported
  - Query decomposition module
    - Breaks up a query into subqueries that can be executed at the individual sites
    - Strategy for combining results must be generated
  - Catalog stores attribute list and/or guard condition

4.6 Types of Distributed Database Systems



Cont'd…

- **Homogeneous Distributed Databases:**

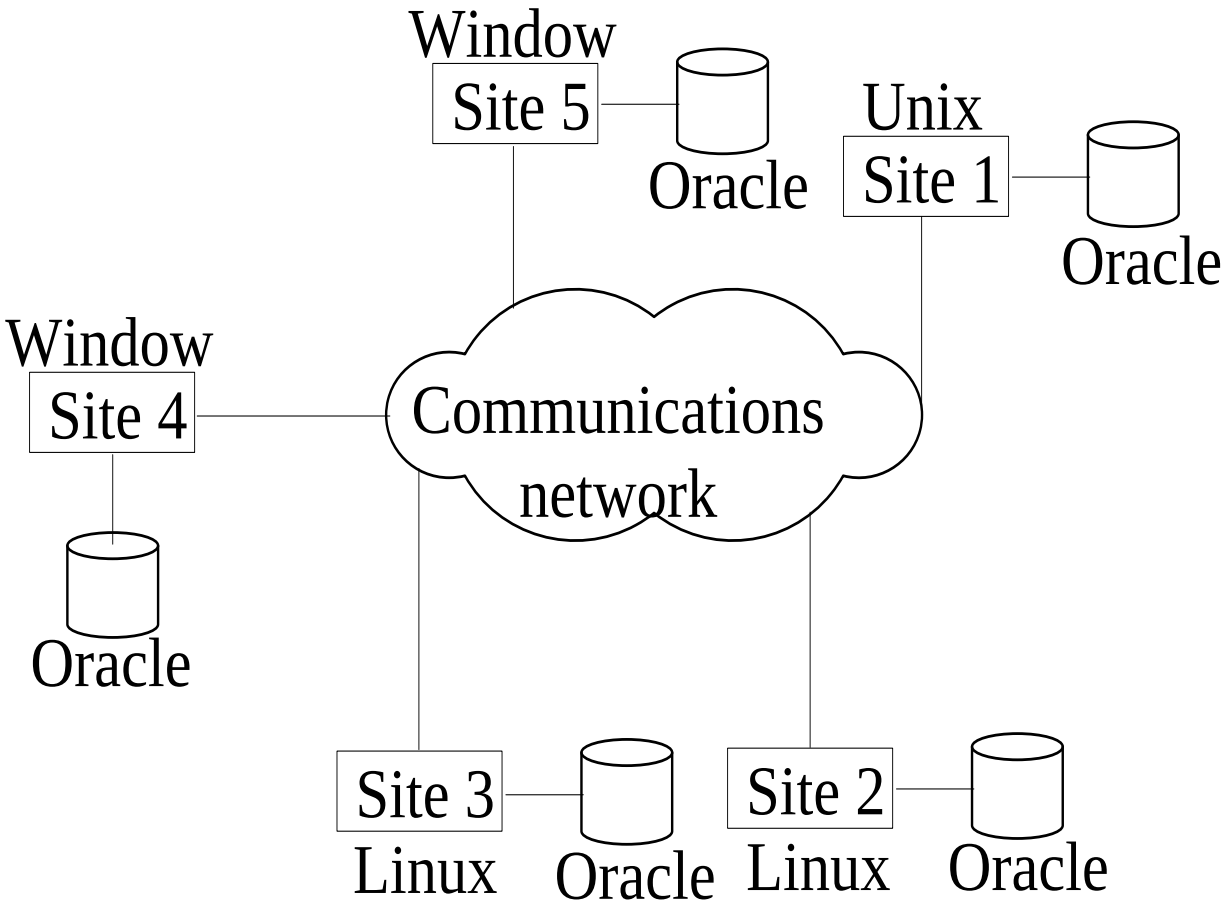


Figure: Homogeneous distributed databases

Cont'd…

- **Heterogeneous Distributed Databases:**

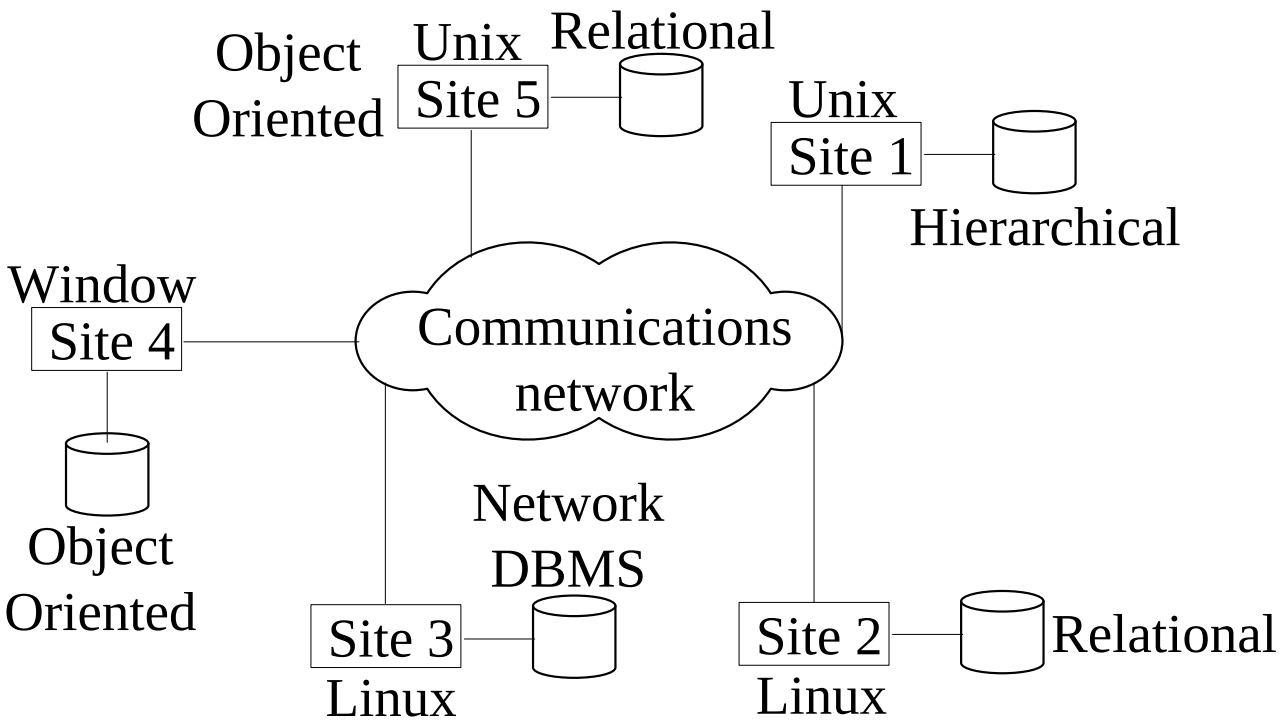
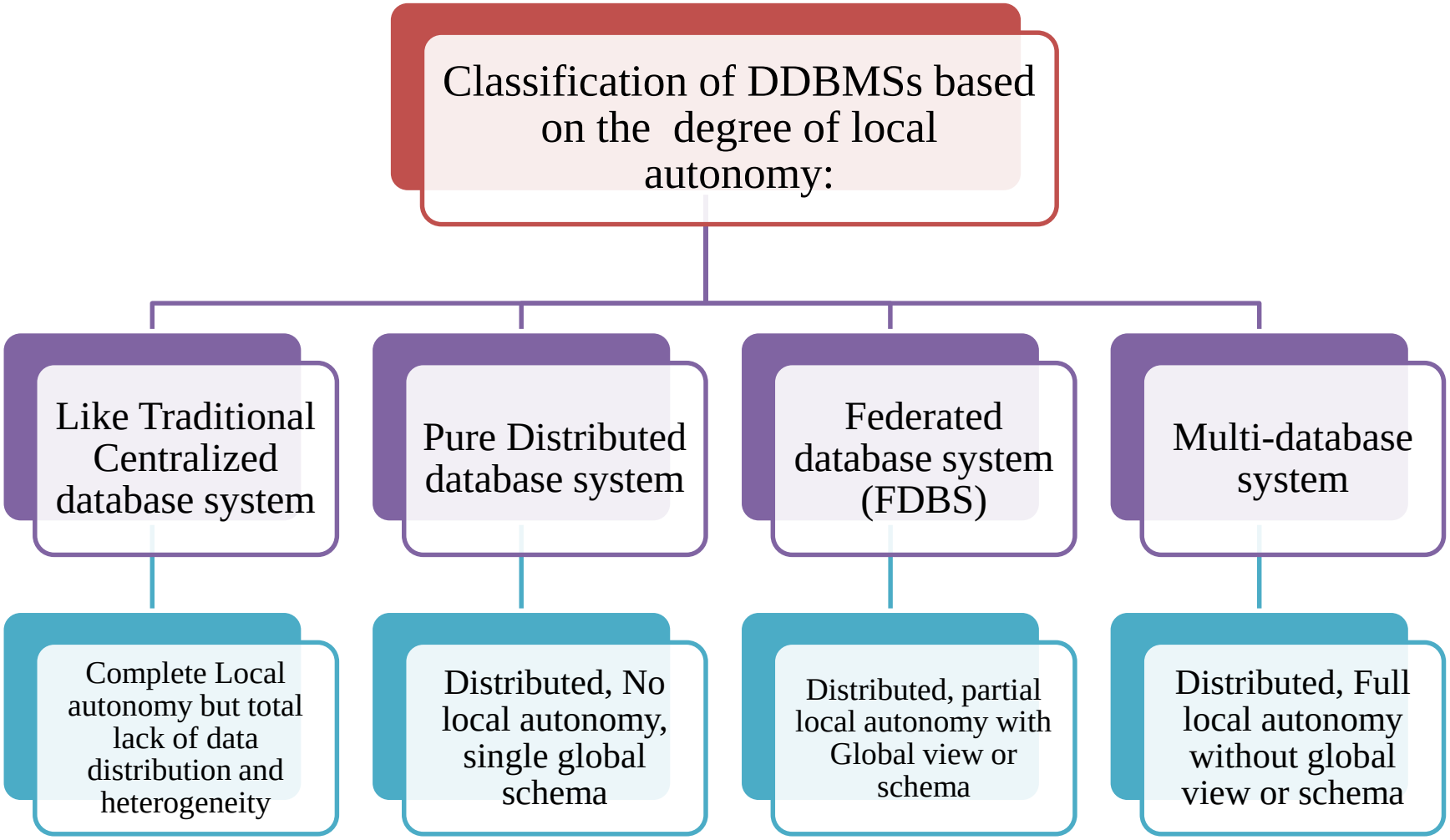


Figure: Heterogeneous distributed databases

Cont'd…


Classification of DDBMSs based on the degree of local autonomy:
Like Traditional Centralized database system
Pure Distributed database system
Federated database system (FDBS)
Multi-database system
Complete Local autonomy but total lack of data distribution and heterogeneity
Distributed, No local autonomy, single global schema
Distributed, partial local autonomy with Global view or schema
Distributed, Full local autonomy without global view or schema

Cont'd…

- **Classification of Distributed Databases:**

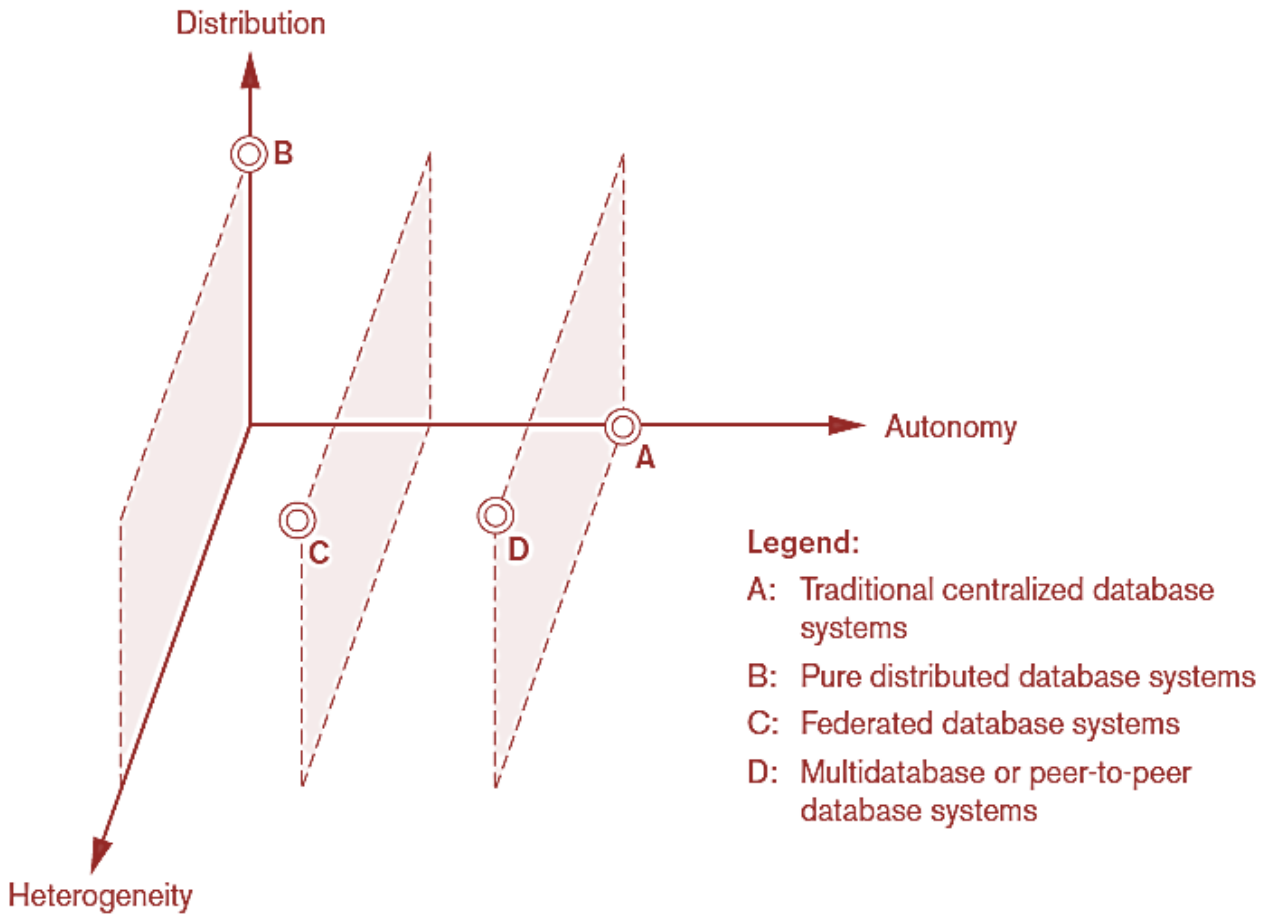
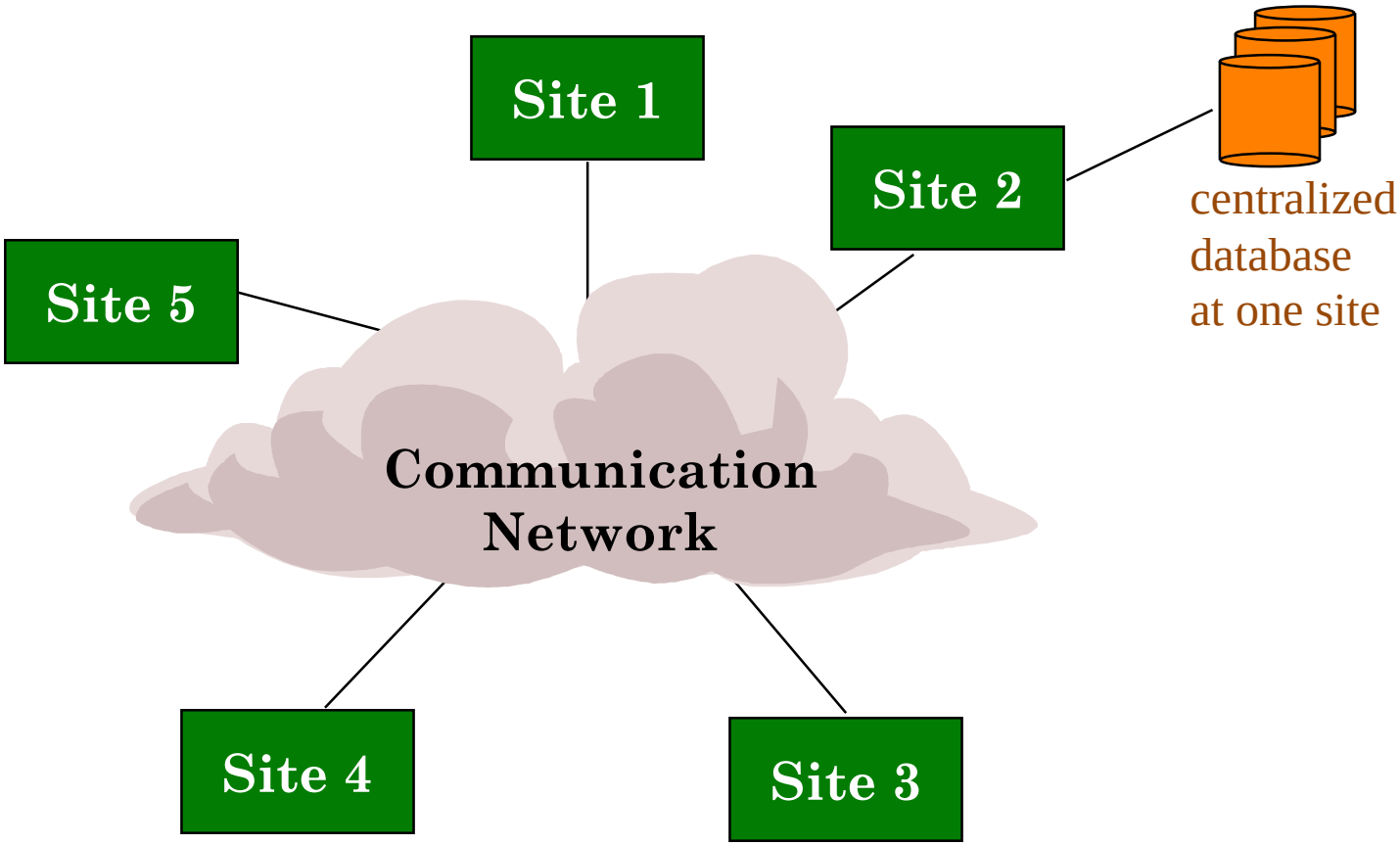


Figure: Classification of distributed databases

Cont'd…

- **A networked architecture with a centralized database at one of the sites :**



Cont'd…

- **Pure Distributed database system:**

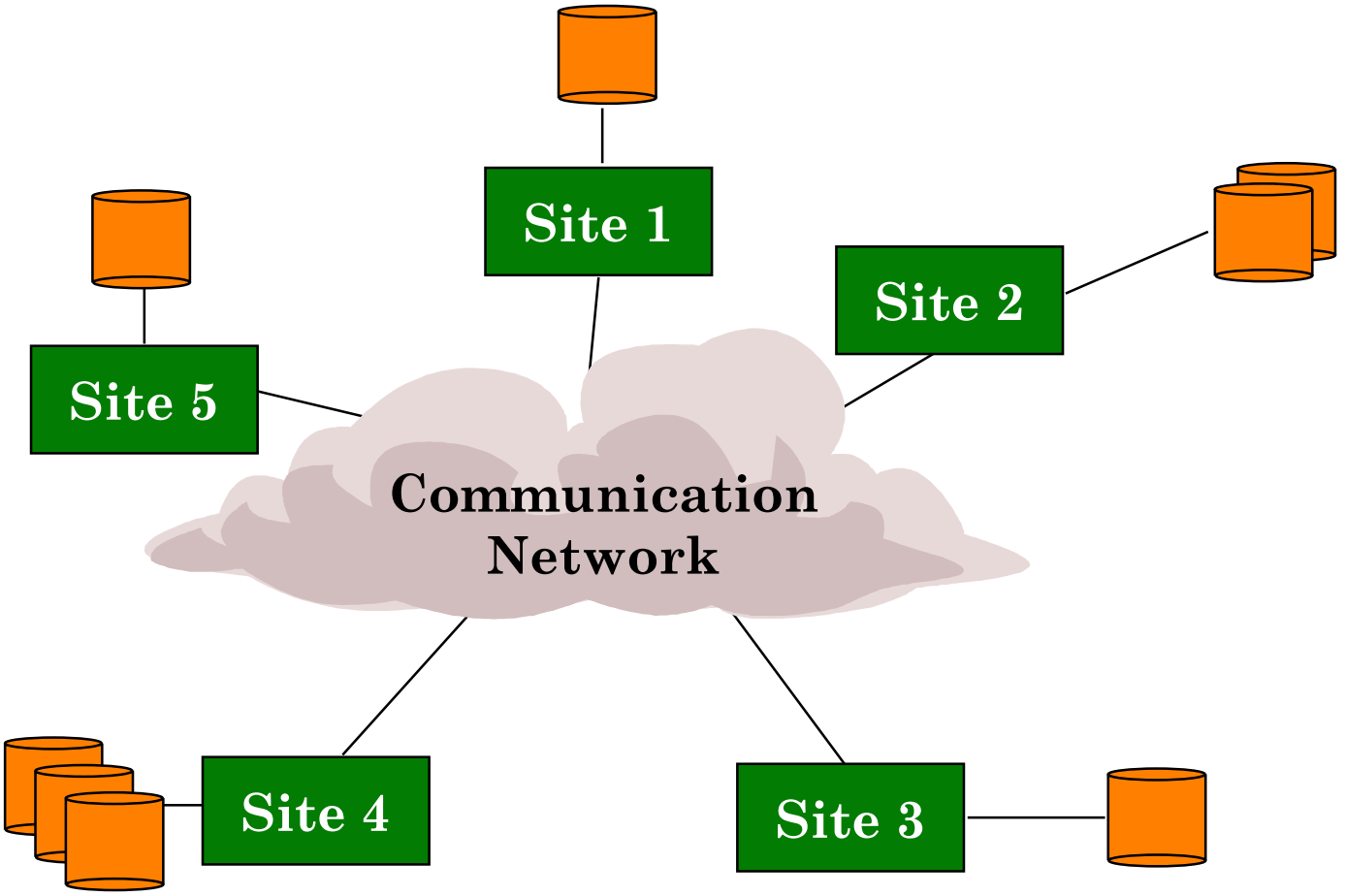
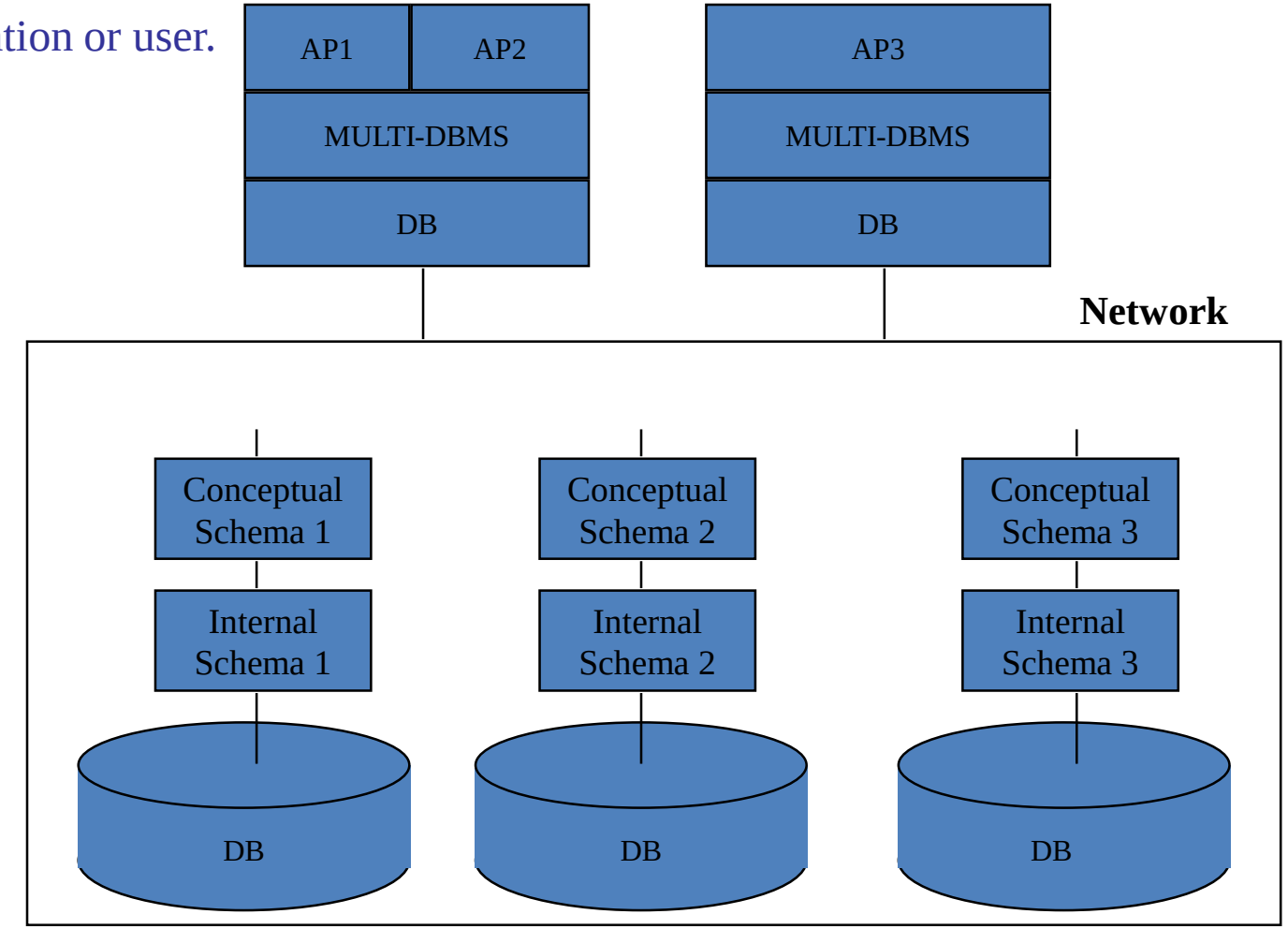


Figure: Pure Distributed Database

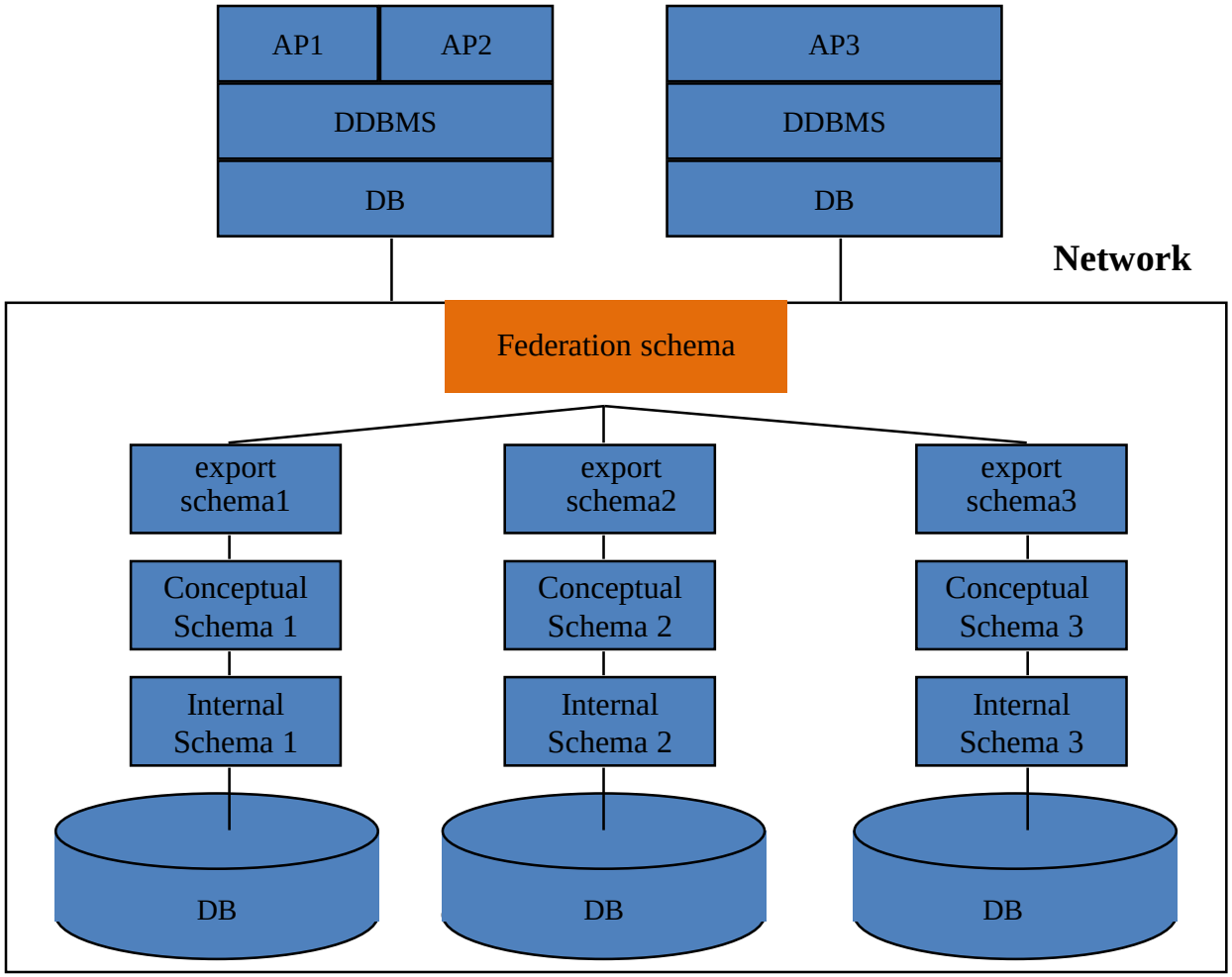
Cont'd…

- **Multi Database or Peer to peer system:** There is no one conceptual global schema. For data access a schema is constructed dynamically as needed by the application or user.



Cont'd…

- **Federated Database:** Each site may run different database system but the data access is managed through a single conceptual schema known as federation schema.



Cont'd…

- **4.6.1. Federated database management systems issues**:
  - A federated database is constructed from multiple autonomous as well as heterogenous databases.
  - Its main challenge is to create the global conceptual federation schema by:
    - Resolving heterogeneity and
    - Preserving the local autonomy.

Cont'd…

- **Following heterogeneity issues affects the design of FDBSs:**
  - Differences in data models:
    - Relational, Objected oriented, hierarchical, network, etc.
    - The same data may be represented differently in different databases. E.g. customer vs. client.
    - This makes global query processing difficult.
  - Differences in constraints:
    - Each site may have their own data accessing and processing constraints. E.g. relationship vs integrity constraint
    - Conflicting constraints must be reconciled in the global schema.
  - Differences in query language:
    - Some site may use SQL, some may use SQL-89, some may use SQL-92, and so on.
    - Queries must be translated between systems.

Cont'd…

- **Semantic Heterogeneity:** Differences in meaning, interpretation, and intended use of the same or related data**.** Following autonomy related issues affects the semantic heterogeneity**:**
  - Design autonomy allows definition of the following parameters
    - The universe of discourse from which the data is drawn, Representation and naming
    - Understanding, meaning, and subjective interpretation of data,
    - Transaction and policy constraints, Derivation of summaries
  - Communication autonomy: Decide whether to communicate with another component DBS.
  - Execution autonomy: Execute local operations without interference from external operations by other component DBSs and Ability to decide order of execution.
  - Association autonomy: Decide whether and how much to share its functionality and resources.

4.7 Distributed Database Architectures

- **Parallel versus distributed architectures**:

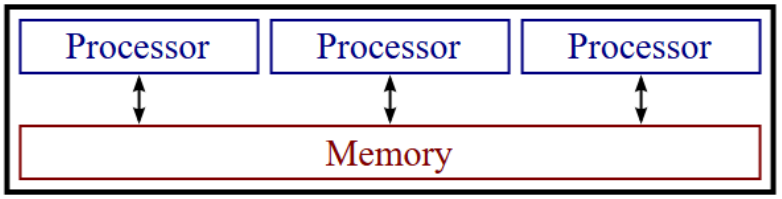


Figure: Parallel System Architecture

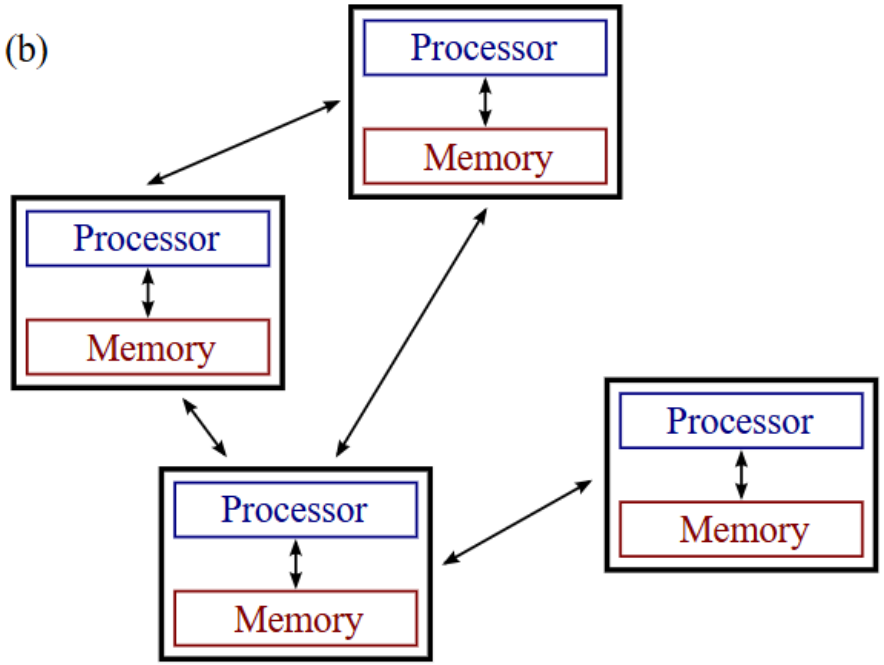


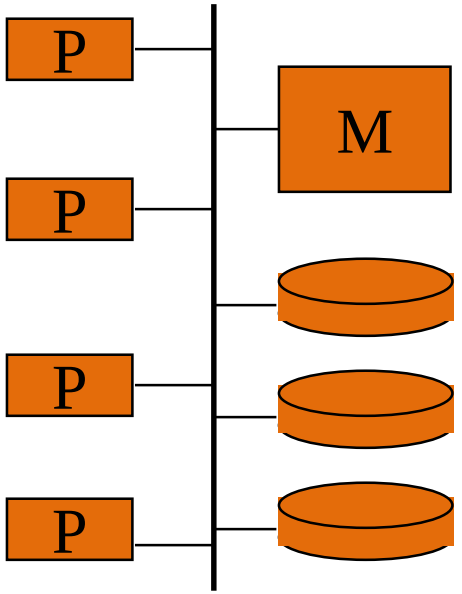
Figure: Distributed System Architectures

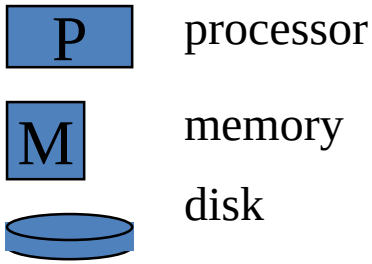
Cont'd…

- **Types of multiprocessor system architectures:** A database in which a single query may be executed by multiple processors working together in parallel.
  - Shared memory (tightly coupled)
  - Shared disk (loosely coupled)
  - Shared-nothing

Cont'd…

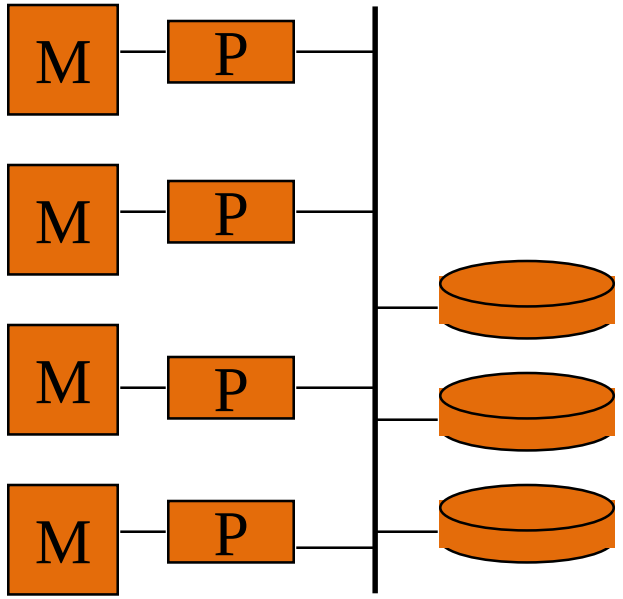
- **Shared memory (tightly coupled):**
  - Processors share memory via bus
  - Extremely efficient processor communication via memory writes
  - Bus becomes the bottleneck
  - Not scalable beyond 32 or 64 processors





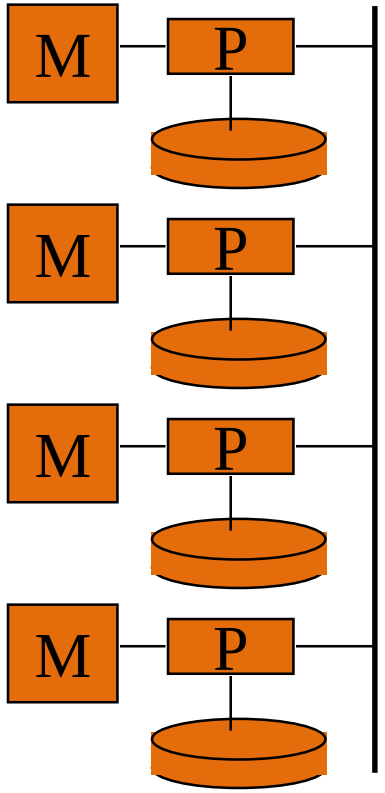
Cont'd…

- **Shared disk (loosely coupled):**
  - Processors share disk via interconnection network
  - Memory bus not a bottleneck
  - Fault tolerance with respect to processor or memory failure
  - Scales better than shared memory
  - Interconnection network to disk subsystem is a bottleneck
  - Used in ORACLE RDB



Cont'd…

- **Shared-nothing:**
  - Scales better than shared memory and shared disk
  - Main drawbacks:
    - Higher processor communication cost
    - Higher cost of non-local disk access
  - Used in the Teradata database machine



Above types of architectures mainly used for parallel DBMSs rather than DDBMSs, since they utilize parallel processor technology.

Cont'd…

- **General Architecture of Pure Distributed Databases**:

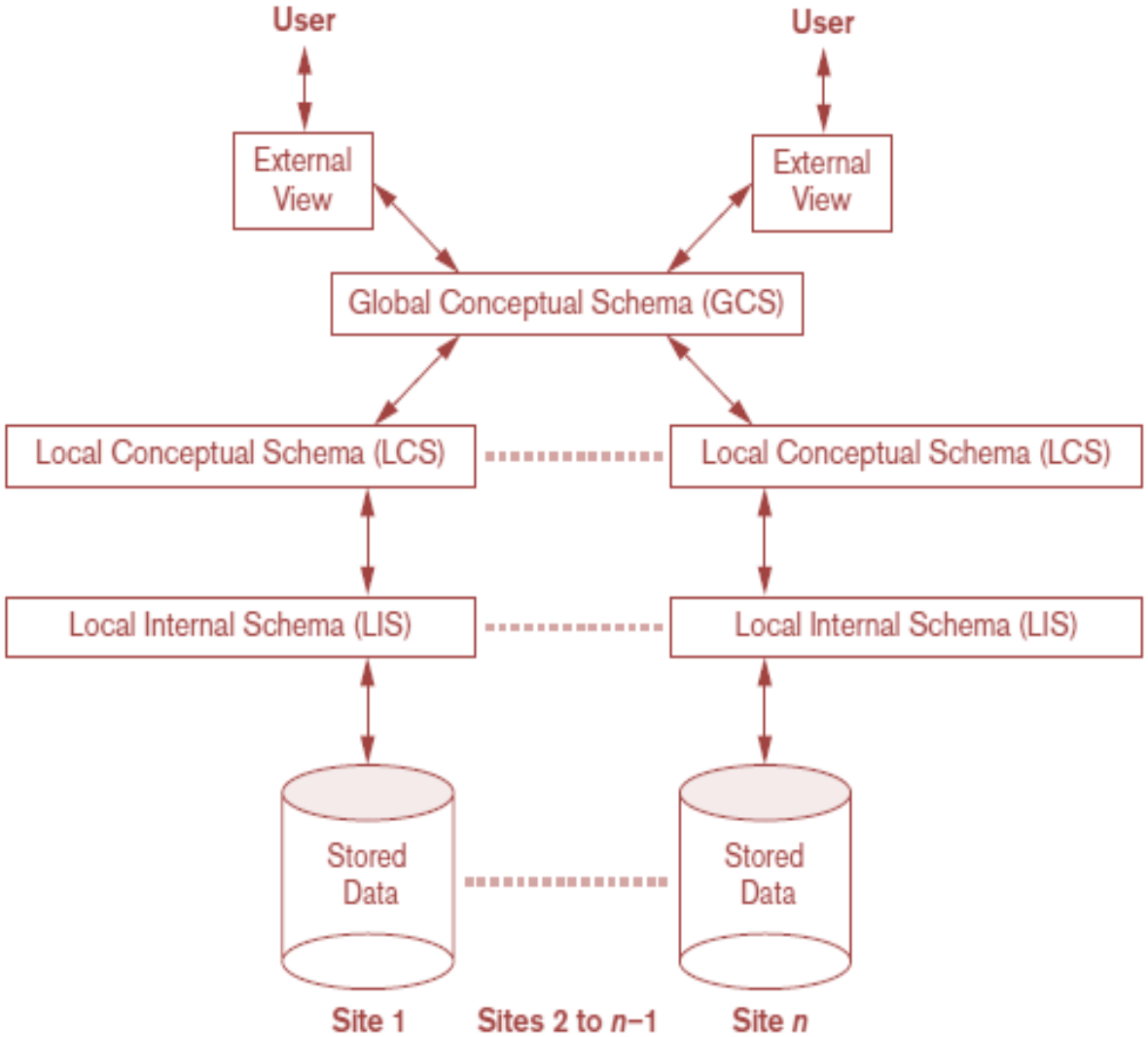


Figure: Schema or component architecture of distributed databases

Cont'd…

- A Pure Distributed Database architecture provides users with a single logical view of data through the Global Conceptual Schema (GCS).
- Each site maintains its own Local Conceptual Schema (LCS) and Local Internal Schema (LIS).
- The GCS and LCS mappings provide fragmentation and replication transparency.
- Global query compiler: References global conceptual schema from the global system catalog to verify and impose defined constraints
- Global query optimizer: Generates optimized local queries from global queries
- Global transaction manager: Coordinates the execution across multiple sites with the local transaction managers

Cont'd…

- **Federated Database Schema Architecture:**

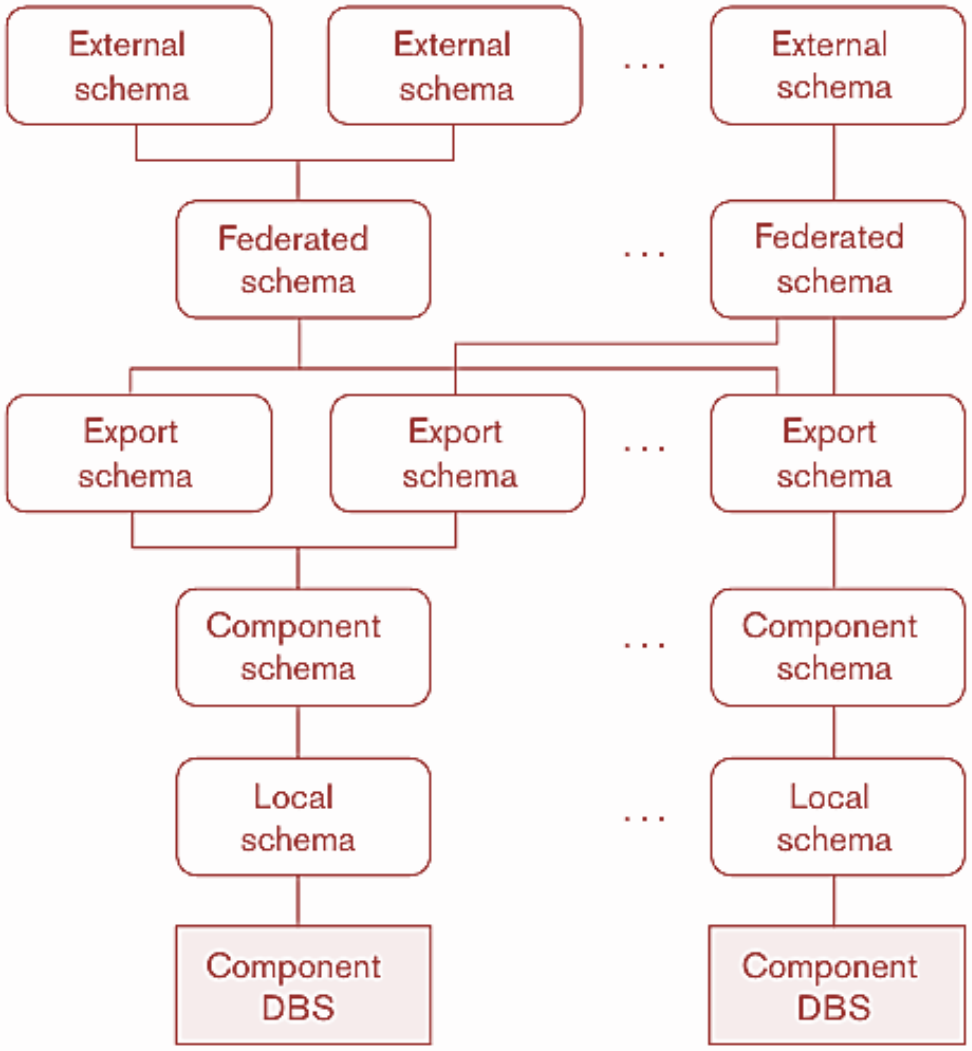


Figure: The five-level schema architecture in a federated database system (FDBS)

Cont'd…

- Federated Database Schema Architecture uses a five-level schema model consisting of:
  - Local Schema: The local schema represents the complete local database
  - Component Schema: The component schema translates it into a common data model
  - Export Schema: The export schema defines the shareable data
  - Federated Schema: The federated schema integrates all exported schemas into a global view and
  - External Schema: the external schema provides user-specific views of the federated database.
- This architecture supports integration of heterogeneous and autonomous databases while preserving local autonomy.

Cont'd…

- **An Overview of Three-Tier Client/Server Architecture:** Division of DBMS functionality among the three tiers can vary

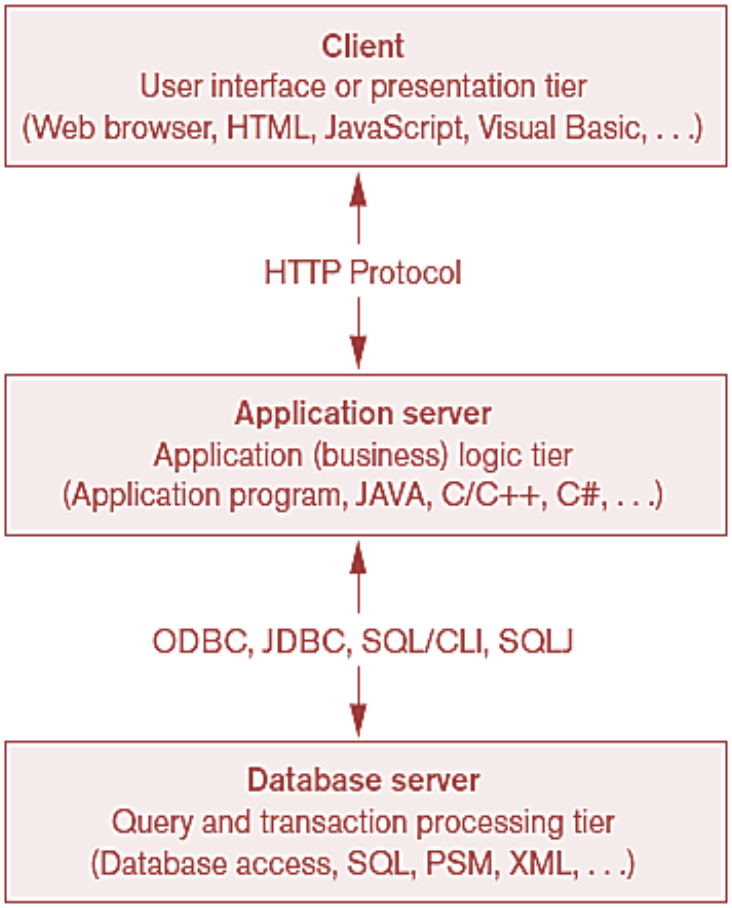


Figure: The three-tier client/server architecture

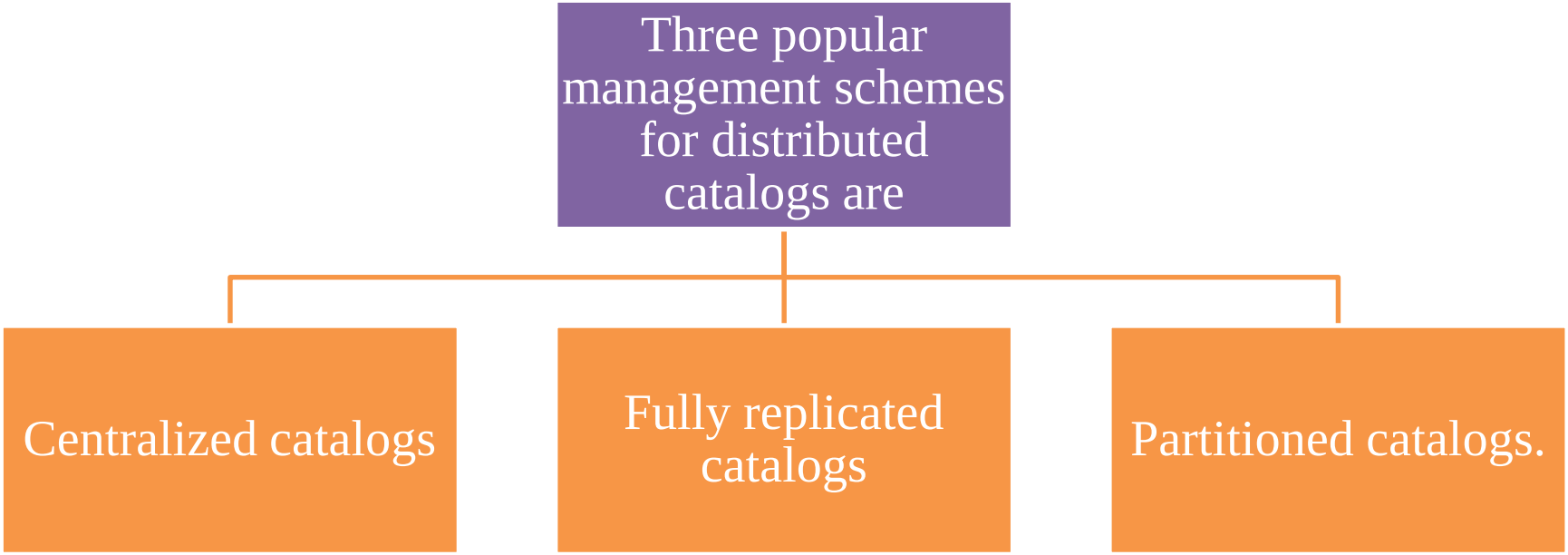
Cont'd…

- A three tier client server architecture Consists of:
  - Presentation Layer: provides the user interface and handles user interaction through web or client applications.
  - Application Layer: contains the business logic, formulates and decomposes global queries into site-specific local queries, communicates with database servers, combines results from multiple sites, and manages security, consistency, distributed transaction coordination, and execution planning.
  - Database Server Layer: processes local queries, performs data storage and retrieval, executes SQL operations, and returns results to the application layer, which then formats and delivers the final output to the client.

4.8 Distributed Catalog Management

- Catalogs are databases themselves containing metadata about the distributed database system.
- Efficient catalog management in distributed databases is critical to ensure satisfactory performance related to:
  - Site autonomy,
  - View management, and
  - Data distribution and replication.

Cont'd…


Three popular management schemes for distributed catalogs are
Centralized catalogs
Fully replicated catalogs
Partitioned catalogs.

Cont'd…

- Centralized catalogs
  - Entire catalog is stored at one single site
  - Easy to implement
- Fully replicated catalogs
  - Identical copies of the complete catalog are present at each site
  - Results in faster reads
- Partially replicated catalogs
  - Each site maintains complete catalog information on data stored locally at that site

Thank You !